

Solution Design Document
For Allied Bank Integration Project

Document Control

Owner: Visa-Cybersource Implementation Specialists

Date	Notes	Version
July 13th 2025	1st. Release	1.0

Contents

1.	Introduction.....	5
1.1.	Notice of confidentiality.....	5
1.2.	Disclaimer:.....	5
1.3.	Audience and Purpose	5
1.4.	Prerequisites.....	5
2.	Test Accounts (EBC).....	6
2.1.	REST API Authentication.....	6
2.2.	Hosts.....	6
3.	Cybersource Services Per Flow	7
3.1.	Flows' Table	7
3.2.	Notes for the technical flow.....	7
3.3.	Flex Microform Flow for Card Payments.....	8
3.4.	Cybersource REST Backends used for this project.....	8
4.	Flex Microform	9
4.1.	Overview	9
4.2.	Flex Microform Flow.....	9
4.3.	Flex Microform Capture Context.....	10
4.4.	Verify Capture Context Signature.....	12
4.5.	Rendering the Microform:	12
4.6.	Transient Token Sample	13
4.7.	Verify transient JWT Token Signature	14
5.	Payer Authentication	15
	Payer Authentication Flow.....	15
5.1.	Step 1 Payer Authentication setup service.....	15
5.1.1.	PA-Setup Service Request data.....	15
5.1.2.	PA-Setup Service Response.....	16
5.2.	Step 2 Device Data Collection.....	17
5.2.1.	Create Device Data Collection iframe.....	17
5.2.2.	Invoke Device Data Collection Iframe	18
5.2.3.	Capture Result of Device Data Collection Attempt.....	18
5.2.4.	DDC Backup	19
5.3.	Step 3 Payer Authentication Check Enrollment	19
5.3.1.	Payer Authentication Check Enrollment request fields	20
5.3.2.	PA Enrollment Check Response data.....	23
5.4.	Step 4 Step-up Challenge Iframe.....	27
5.4.1.	Create Challenge Request Iframe.....	28

5.4.2.	Invoke Challenge Request Iframe	28
5.4.3.	Capture Result of Challenge Attempt	29
5.5.	Payer Authentication Validation.....	29
5.5.1.	Payer Authentication Validation Request	29
5.5.2.	PA-Validate Response data	31
6.	Payments	35
6.1.	Authorization + Capture +TMS/NT	35
6.2.	Card Verification Using Zero Authorization.....	40
6.3.	Separate Capture for Payments.....	40
7.	Reverse Payments	42
7.1.	Authorization Reversal	42
7.2.	Void a Payment.....	43
7.3.	VOID a Capture	44
7.4.	Refund a Payment.....	45
7.5.	Refund Capture	47
7.6.	Bulk Transactions.....	48
8.	Transaction Search.....	50
8.1.	Search using query	50
	Could use the backend /tss/v2/searches to search transactions using query inputs.	50
8.2.	Retrieve transaction details.....	51
	APPENDIX A: Useful Links	53
	APPENDIX B: Configure test MID in Postman	54
	APPENDIX C: Data Types	55
	APPENDIX D: MID Key Handling	56
	APPENDIX E: Billing Default Data.....	57
	APPENDIX F: Connect to Cybersource	58

1. Introduction

This document describes how to implement Cybersource integration for card payments.

1.1. Notice of confidentiality

Important – please read.

This document is furnished to you solely in your capacity as a customer of Visa Inc. and/or a participant in the Visa payments system. By accepting this document, you acknowledge that the information contained here is confidential and subject to the confidentiality restrictions contained in Visa's operating regulations and/or other confidentiality agreements, which limit your use of the Information. You agree to keep the Information confidential and not to use the Information for any purpose other than in your capacity as a customer of Visa Inc. or as a participant in the Visa payments system. The Information may only be disseminated within your organization on a need-to-know basis to enable your participation in the Visa payments system. Please be advised that the Information may constitute material nonpublic information under U.S. federal securities laws and that purchasing or selling securities of Visa Inc. while being aware of material nonpublic information would constitute a violation of applicable U.S. federal securities laws.

1.2. Disclaimer:

Important – please read.

Any XML or json examples provided in this document are for guidance only and in no way represent the full data set you should be passing during the payment processing with Cybersource.

Official Cybersource documents are the considered reference in case of contradictions between this document and Cybersource official documents.

Make sure all data relevant to your organization is included during testing.

1.3. Audience and Purpose

This guide was written for Allied Bank integration project to accept payments.

1.4. Prerequisites

You must be able to create web pages that will gather customer and order information (Billing and Shipping information excluding card data) for payment and fraud screening services. This data needs to be included in requests to REST API Payment Endpoint, and you must be able to process the response information to fulfil the customer's order.

2. Test Accounts (EBC)

To utilize the CyberSource payment gateway services, all merchants are required to obtain and activate a CyberSource Merchant ID (Organization ID) for integration, testing & live transactional processing.

The Cybersource Enterprise Business Centre (EBC) is a customer portal provision to merchants to allow merchants to view transactions (Test/Live) and perform other functions. EBC has a Test & Live Environments. Test EBC will display test transactions sent to test endpoints and Live EBC will display live transactions sent to live endpoints.

- Test EBC Environment <https://ubctest.cybersource.com/ebc2/>
- Live EBC Environment: <https://ubc.cybersource.com/ebc2/>

Cybersource Merchants ID's (Test/Live) will be configured with the appropriate payment methods, currencies, and payment processor(s). Additionally, value-added services such as Tokenization, and Fraud Management Essentials if required.

2.1. REST API Authentication

For REST API authentication, the Merchant needs to generate REST Security key, which comes in two types:

- Shared secret key for using HTTP Signature authentication
- P12 certificate for using JSON Web Token authentication

Please select an authentication key that meets your integration method:

Please refer to the developer portal link below for additional information on Authentication and generating headers.

[Authentication \(cybersource.com\)](#)

For more information on how to generate a Secret Key & P12 Certificate for REST API authentication please see below:

[Create a Shared Secret Key for HTTP Signature Authentication](#)

[Create a P12 Certificate for JSON Web Token Authentication](#)

2.2. Hosts

Cybersource has two hosts listed below:

Sandbox API Endpoint: <https://apitest.cybersource.com>

Production API Endpoint: <https://api.cybersource.com>

3. Cybersource Services Per Flow

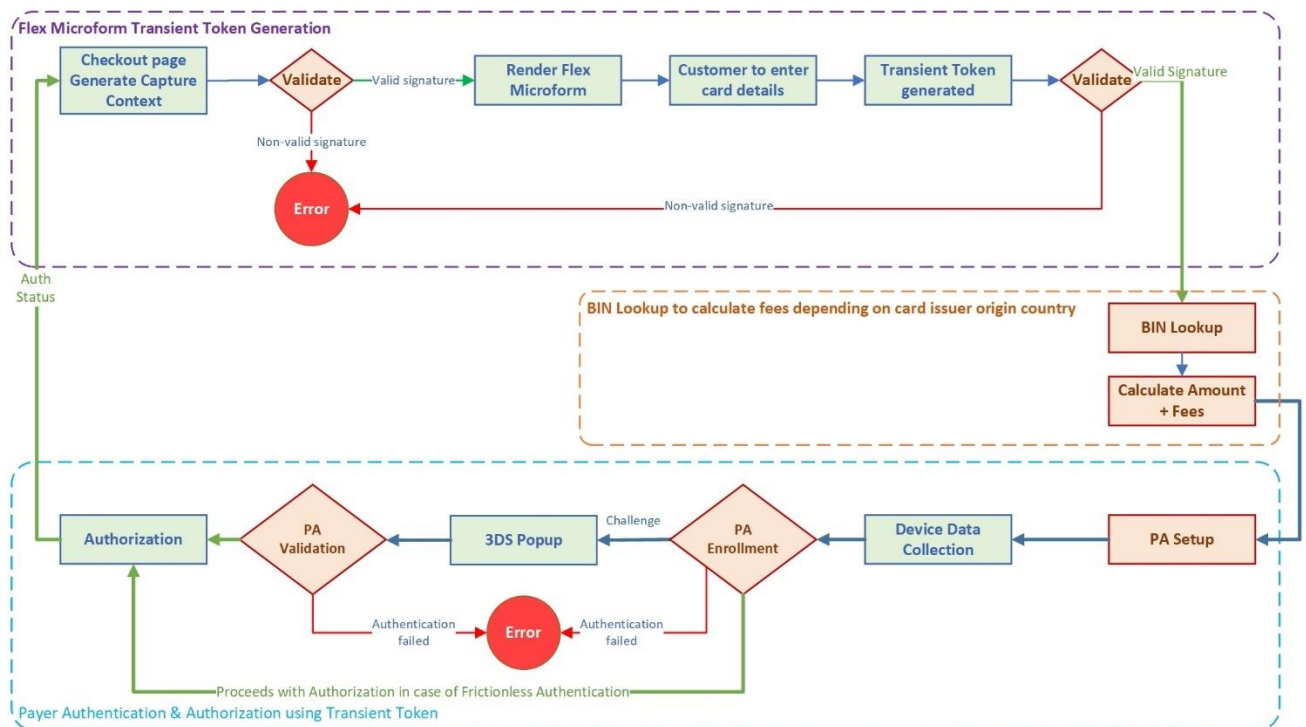
3.1. Flows' Table

Flow	Services (In Order)
1 Generate Transient Token for card payments Flex Microform is a secure payment for non-PCI compliant merchants	• Generate Capture Context • GET public key • Verify Capture Context Signature • Render Microform • Verify Transient Token Signature
2 CIT Payment using Transient Token	• PA-Setup • Device Data Collection • PA-Enrollment check • OTP challenge (Issuer decision) • PA-Validate • Authorization + DM + Capture
3 CIT Payment using card	• PA-Setup • Device Data Collection • PA-Enrollment check • OTP challenge (Issuer decision) • PA-Validate • Authorization + DM + Capture
4 Card Verification using zero Auth	• PA-Setup • Device Data Collection • PA-Enrollment check • OTP challenge (Issuer decision) • PA-Validate • Authorization + DM
	•
5 Capture Payment	• Capture
6 Reversals, VOIDs, and Refunds	• Reverse, VOID, Refund APIs
7 Retrieve transaction details	• Retrieve transaction details
8 Search transactions	• Search transactions

3.2. Notes for the technical flow.

- PCI compliant merchants could use card details with REST APIs for payments and PA.
- NON-PCI compliant merchants to use Transient Token will be used for payment and PA.
- Payer Authentication could be done separately from Authorization or could be combined with Authorization
- Payment flow in this document to depend on separate calls of Payer Authentication and Authorization

3.3. Flex Microform Flow for Card Payments



3.4. Cybersource REST Backends used for this project

Name	Services	URL	Type
1 Payment	- Authorization - PA-Enrolment - PA-Validate - Capture - DM - Tokenization	/pts/v2/payments/	POST
2 Payer Authentication Setup	PA Setup	/risk/v1/authentication-setups	POST
3 PA Check Enrollment	PA-Enrollment	/risk/v1/authentications	POST
4 PA Validation	PA-Validate	/risk/v1/authentication-results	POST
6 Capture	Capture	/pts/v2/payments/{id}/captures	POST
7 Transaction Details	Retrieve transaction logs	/tss/v2/transactions/{id}	GET
8 Search transactions	Search transaction	/tss/v2/searches	POST
9 Authorization Reversal	Auth reverse	/pts/v2/payments/{id}/reversals	POST
10 Void Payment	Void combined payment	/pts/v2/payments/{id}/voids	POST
11 Void Capture	Void Capture	/pts/v2/captures/{id}/voids	POST
12 Refund Payment	Refund combined payment	/pts/v2/payments/{id}/refunds	POST
13 Refund Capture	Refund Capture	/pts/v2/captures/{id}/refunds	POST

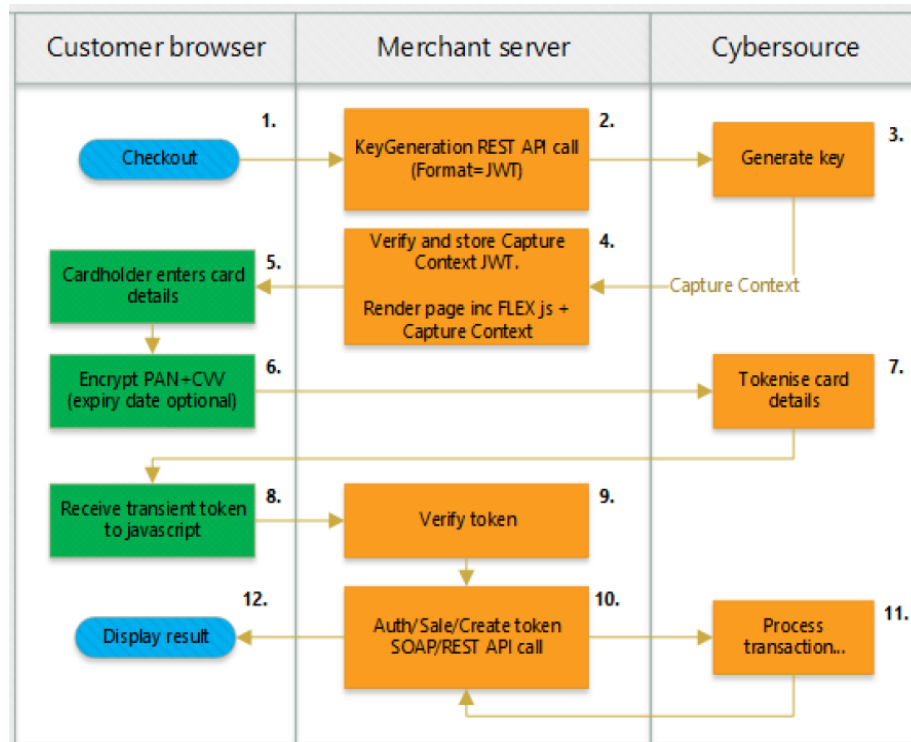
4. Flex Microform

4.1. Overview

- Flex Microform provides the most secure method for tokenizing sensitive card data by replacing the PAN and CVN fields with Smartpay Fuse-hosted iframes and tokenizes these payment card details on your behalf.
- Sensitive data is encrypted on the customer's device before HTTPS transmission to Smartpay Fuse which reduces the potential for man-in-the middle attacks on the HTTPS connection.
- Use of Flex Microform can help qualify you for the minimum PCI DSS scope (SAQ A) as the Authorization uses the token rather than card details.
- You can style these fields to look and behave like any other field on your website.
- Please note, that if you want to use Flex Microform in a mobile app it is necessary to place it in a WebView. This is the recommend approach.
- When the cardholder has keyed in their card details, Flex Microform will return a token to your page's JavaScript, which can safely be sent to your server for use in place of the card details, in payment transactions using REST APIs.
- The token returned is short-lived or "transient". This means that the token will expire after 15 minutes, or the first authorization, whichever is the soonest. After this, it can no longer be used.
- The transient token can be used to create a permanent TMS Token in the same API call as the authorization request, if you want to implement a "returning customer" payment flow.

4.2. Flex Microform Flow

1. Customer is redirected to the payment page with Microform integration.
2. Merchant server requests Capture Context from Smartpay Fuse.
3. Smartpay Fuse generates Microform Capture Context.
4. Merchant verifies JWT received from Smartpay Fuse and sends Checkout page (including Capture Context) to the browser.
5. Customer enters card details
6. PAN and CVN (and optionally expiry date) are encrypted in the browser by the Microform.
7. Encrypted card details are sent directly to Smartpay Fuse for tokenization.
8. Transient token is received in the browser (in the form of a JWT) and forwarded to the merchant server for verification.
9. Merchant verifies Token.
10. Merchant includes Token in Authorization/Sale/PA SOAP/REST API request
11. Cybersource processes transaction and returns result to the merchant server
12. Merchant server displays the result of the payment to the customer



Please check this video <https://www.youtube.com/watch?v=-qv7nzPvW-M>

4.3. Flex Microform Capture Context

The first step in integrating with Microform is requesting a digitally signed JWT capture context from Cybersource.

Capture context is requested by Flex API, which is a simple API that allows for multiparty communication model, where one party sends (usually from the front-end, e.g. cardholder's browser or mobile app) information to Cybersource in exchange for a token.

The other party (usually a merchant server) receives the token directly from the front-end and uses it later.

Data the token represents is kept securely in Cybersource.

Capture Context is signed so that the receiving party can verify it is coming from Cybersource.

Sample REST Request:

</microform/v2/sessions>

```

{
  "targetOrigins": [
    http://localhost:8080 //use your domain
  ],
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD"
  ],
  "clientVersion": "v2"
}

```

Response:

Decoded Capture Context

```
{
  "kid": "zu",
  "alg": "RS256"
}
```

```
{
  "flx": {
    "path": "/flex/v2/tokens",
    "data": "EBgZWw/ZI1CsKRETH0zCxAAEGzcbSFD3ZT7fohcM4CRvE2HLxYrJl2kRC1jkdGej35si9F53HKF2i03oKHpjS1+JA54H8nG8Q7W/eYF2ILOX7M=",
    "origin": "https://testflex.cybersource.com",
    "jwk": {
      "kty": "RSA",
      "e": "AQAB",
      "use": "enc",
      "n": "yJjRFG58ihrxjtIF7x8httYw0ZrgAcFlgC1W2fyN18zFuvpD3y1k1ci5GFf5pgwShjncqHnSePA1180h4G8yAadwKosoAFJcaVHYNPre-UZWoBoUQwLIXM87oy0v5KRfKt-p0dit98WasIbYnHJfJmmGYfyi12ny1KS15n6WL12qVRRFvNwYYF3rUuHs_CSPsvAqi0voUHqhMBtXMTfqQZbF1uqhH5jJ2reuAgI0pxDqc3jXq9bpV00q7VA5MnbgsK30BfD60dktUd92S6PiCRLSuXzadmiS9AEcoZ9MdhzIixMGAqNGT1Dhq7fwx7yCXLEXFbG1NHVjQMv2gw",
      "kid": "08E01vN0HxqUVHE7WebRgd9Kpw1xeXqS"
    }
  },
  "ctx": [
    {
      "data": {
        "clientLibraryIntegrity": "sha256-H/fByAbbp4aRmpfXYT6VyWJsYb7T/f7MJwEyU46ie8k=",
        "clientLibrary": "https://testflex.cybersource.com/microform/bundle/v2.5.3/flex-microform.min.js",
        "allowedCardNetworks": [
          "VISA",
          "MASTERCARD"
        ],
        "targetOrigins": [
          "http://localhost:8080"
        ],
        "mfOrigin": "https://testflex.cybersource.com",
        "allowedPaymentTypes": [
          "CARD"
        ]
      }
    },
    {
      "type": "mf-2.1.0"
    }
  ]
}
```

```
    }  
  ],  
  "iss": "Flex API",  
  "exp": 1743941527,  
  "iat": 1743940627,  
  "jti": "GCT3cY0KdenycF8h"  
}
```



IMPORTANT:

- Capture Context is called while loading the checkout page
- Request to generate capture context must be sent from secure server.

4.4. Verify Capture Context Signature

By decoding Capture Context, the KID in the header should be used to get the public key to validate capture context signature.

The public key to be retrieved from the GET request to the end point </flex/v2/public-keys/> by passing the key ID (kid) that you obtained from the capture context header.

Sample GET request using {kid}=zu <https://apitest.cybersource.com/flex/v2/public-keys/zu>

The response is:

```
{
  "kty": "RSA",
  "use": "enc",
  "kid": "Zu",
  "n": "0zmvkUgZwNHs9cEc5PwWbG-dmSjPcoQFxEbqH_fBjkj_nfTTKshdiSq5ciulWEa_rrqQ2qwcSADNxtTzRR1qfud-NvsM8V1tT7xDuVVqPTZoWLKa0BWxgQQ-1mCm1KdGltYwccB0R1LoF-rb3DEEZySsHvqErYzYt4M_rqjEiK5Y9y1h3k1h5Yk4zGLWchko3jiDS-pVevvWsQsN-Y3KuB19485G9P_MXLtFjWQ4wC4jloetdD_hgDfxx-hQy3wuwHfHfLdxvxiB8X5Is4m6DuY4_7hS5RwXAJ01Qsd-zUYZNT_2yWVR56_jyiZEiOdgiM9QtLPZCTKzqsXoqZQ",
  "e": "AQAB"
}
```

Use the public key to verify Capture Context signature.

Sample signature validation using

<https://jwt.io>

IY30i1q1MUWU5W51aWfWbJ1oXNjK5NUQ5N1A5CLcQ
 dgk0i1JcG5Q5NqYFVdXVY1UldhHJK1nE.-F-
 IXd083d3j2JAKUWNReHjho1BHE9Wd6ZxCBgIY
 BJ1ezf7J1JZyT9CLcas8HTXdn-
 okmkDk0u5R4HJXz2o4312aneOpYtLrn9CDN1Pz
 HwM9A552AM1enSR321GW5HdZKLUgviYPlbbZ
 Qe1R93gzEhU9LHni-uI-
 Q3p3KJqU7k2d8PmoN18h-HuE6VLUKIS_4I5R8-
 rcPdW64E4J0z7kUenJRNv_T0du_6-6jGEoH1JOS
 rcj4XARfam8ojs45dPdW5E_o4scm0JGRGGYrDo9
 Y5f5pCJJKUoX0zR8J1J32N5M3vksy6H0I05690
 NzM_YeLcKjYgYZLZJ-JsxyA

```
},
"iss": "Flex API",
"exp": 1699450409,
"iat": 1699449509,
"jti": "Fpnj5qXuuHaIptrJ"
```

VERIFY SIGNATURE

```
RSASHA256(  
    base64UrlEncode(header) + "." +  
    base64UrlEncode(payload),  
    -zUYZNT_2yWVR56_jy1ZE10dgiM9  
    QtLPZCKtZqsXoqZQ",  
    "e": "AQAB"  
)
```

Private Key in PKCS #8, PKCS #1, or JWK string format. The key never leaves your browser.

Signature Verified

4.5. Rendering the Microform:

Microform can be described as a combination of two things:

- A javascript library, let's call it microform.js
- A Cybersource-hosted payment page 'iframes', designed to look like a text input field, let's call it a Microform page or just Microform.

Microform page would be embedded into merchant's hosted payment page to where normally the input field would be, thus essentially replacing it.

Client-side script

```
<script
src=" [Insert clientLibrary value here]"
integrity=" [Insert clientLibraryIntegrity value here]"
crossorigin="anonymous">
</script>
```



IMPORTANT:

- Ensure that you do NOT hard code V2 values.
- Get the values by decoding Capture Context response for every transaction.
- The iframe will contain the Microform page hosted by Cybersource.
- The only part of this page which is visible to the user is a text input.
- Microform doesn't support custom fonts.
- Full styling and events list in the guide
<https://developer.cybersource.com/api/developer-guides/dita-flex/SAFlexibleToken/FlexMicroform/Styling.html>

4.6. Transient Token Sample

JWT Transient Token

```
eyJraWQioiIwODBHdk5YZHUxbmtqN1ZrWFRnZ0R0WELQWHQwOEhSbCIiImFsZyI6I1JTMjU2In0.eyJkYXRhIjpw7Im51bWJlciI6IjQwMDAwMFhYWFhYWDewOTEiLCJ0eXB1IjoimDAxIn0sIm1zcyI6IkZsZXgvMDgiLCJleHAiOjE2OTk2MTY0NjAsInR5cGUiOiJtZi0wLjExLjAiLCJpYXQiojE2OTk2MTU1NjAsImp0aSI6IjFFMTVUuzlFV0ZZRE9JNzQzUFBUSU40RV1IUVQzRkE3UzBORT1SQUtTWUdCSzI5VFdYMKU2NTRFMTZDQzcyNDciLCJjb250ZW50Ijp7InBheW1lbnRJbmZvcmlhdG1vbiI6eyJjYXJkIjp7Im51bWJlciI6eyJtYXNrZWRYWYw1ZSI6I1hYWFhYWFhYWDewOTEiLCJiaW4iOiI0MDAwMDAifSwic2VjdXJpdH1Db2R1Ijpw7fSwidHlwZSI6eyJ2YXwx1ZSI6IjAwMSJ9fX19fQ.Fje2k5sxG2L9HavRGfLyFLAo9oG7J5ES0n4Ak8yhufPrUzZHGOPEOTM16N_1Zf62VsUwUKrmIEkS1WBC40dn7YGs2575HYhAg9e9ad70cZEHu-F99xswujkB_9WnEmGYoLOF8bJYNjMIJXZEJqng2aaBk00KGaXqqrUzZqW9NeeSluPZKA15CL0Csb4-abG1AMsngv_RDHU8jwuqXw98-8VXFbeeQM3JaKJMeLmB2uLB7rj4z-930F9yVWZt1PrubJ07dWLHaf6tZ_EK7iPtCDQxzYqfshNsJ5L5LN_gS3Zw9-OEBYpu9ik29Tng-TjKDH0SoyVUo1ZW73yXVabA
```

Decoded Transient Token

HEADER:

```
{
  "kid": "080GvNXdu1nkj7VktTggDtXIPXt08HR1",
  "alg": "RS256"
}
```

PAYLOAD:

```
{
  "iss": "Flex/08",
```

```

"exp": 1743942465,
"type": "mf-2.1.0",
"iat": 1743941565,
"jti": "1E16563845MXMQS7ZU23ELI29LGP7QRWSBE8T3U2JSN1405NIARE67F2734108CA",
"content": {
  "paymentInformation": {
    "card": {
      "expirationYear": {
        "value": "2026"
      },
      "number": {
        "detectedCardTypes": [
          "001"
        ],
        "maskedValue": "XXXXXXXXXXXX1091",
        "bin": "400000"
      },
      "securityCode": {},
      "expirationMonth": {
        "value": "01"
      }
    }
  }
}

```

4.7. Verify transient JWT Token Signature

Get the public key by decoding the JWT Capture Context to get "jwk" field

```

{
  "flx": {
    "path": "/flex/v2/tokens",
    "data": "3gQHRRyzV1xtNeMdCTciUhaAENVW6tRNhbAHznhrECdV9N3y+zEunQpbFZTxf0G6FcLDKG63f294ucsR2CTQ85EuIjE0i8oUZUTKLnDfoiGttBA=",
    "origin": "https://testflex.cybersource.com",
    "jwk": {
      "kty": "RSA",
      "e": "AQAB",
      "use": "enc",
      "n": "jetNCJ7A0r_42-p58IRDEIcCSH8iUB_UQ1GLDYOU_DtHgbrdIE20s6lQ7ht3ZSeHL3Ba_T6JjWes3ti8_ydIeLW6VQ733HOTGUAZ4nMnKHXYgKrhF0RAXYZr06Pr6bKeARDueNnTFiBx9fye-rn9hTt3lYU9Ud3YICgWZPl9Cck2IfEt0xecZmv0yAeQ3FJlvkRuJhnlcLLXuhTkb9icVW21CrNk8T9pw0yHJTstNhauxsz0ZvWePzeuh4v7_bZX6qIz1B4KMPJb3ILRQvie5bscJzilY3j0He1NAeKeUUZBCGFQ3nNNPlD-XDrIVAC2I-WLSFycEBefDhluQdTow",
      "kid": "081Pa5EvilaEk2KjwkQaojWiaDrOfsz6"
    }
  },
  "ctx": [
    {
      "data": {
        "clientLibrary": "https://testflex.cybersource.com/microform/bundle/v1/flex-microform.min.js",
        "targetOrigins": [
          "https://localhost.com"
        ],
        "mfOrigin": "https://testflex.cybersource.com"
      },
      "type": "mf-1.0.0"
    }
  ],
  "iss": "Flex API",
  "exp": 1699450409,
  "iat": 1699449509,
  "jti": "Fpnj5qXuuHaIptrJ"
}

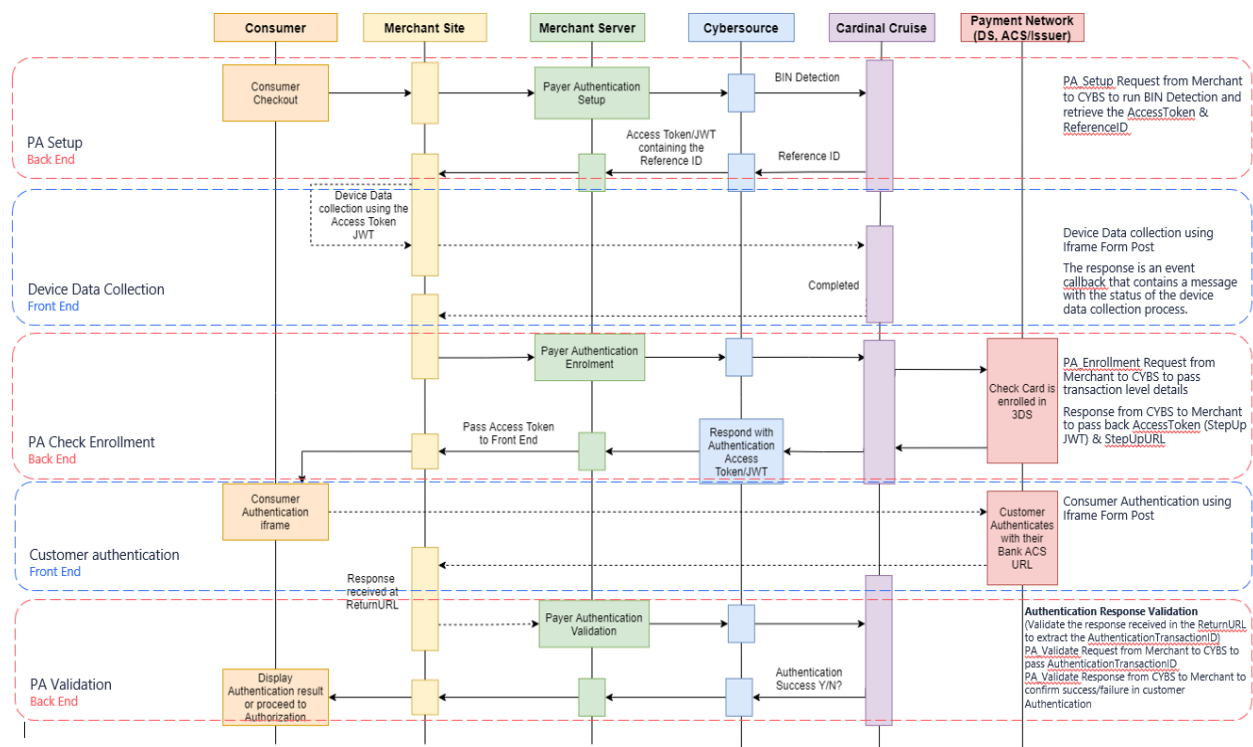
```

Use the key to verify the transient token signature.

5. Payer Authentication

This section describes the steps to perform Payer Authentication Cardinal direct API PA 3DS 2.x Payer Authentication (PA) provides an additional level of security for online payments, whereby cardholders who are enrolled into 3DS provide OTP to complete online transactions. 3DS 2.x eliminates all the disadvantages of the 3DS 1.0. It collects ten times more data during the process, allowing the issuer to better determine if the customer is risky or not. Please check PA developer guide here [Payer Authentication Developer Guide](#)

Payer Authentication Flow



5.1. Step 1 Payer Authentication setup service

Call the Payer Authentication Setup service on the server side after the cardholder has entered their PAN and Expiry date. Therefore, the Payer Authentication Setup service must occur separately without including other services (like the payment endpoint)

5.1.1. PA-Setup Service Request data

When requesting the Payer Authentication Setup service, you must send either the customer's card

More examples can be found on the developer portal [API Reference Assets | CyberSource Developer Center](#)

PA-Setup POST payload examples below

```
{
  "clientReferenceInformation": {
    "code": "9aa45838-ea02-4ba1-8066-47b55431b452"
  },
  "paymentInformation": {
    "card": {
      "type": "001",
      "number": "4000xxxxxxxxx2503",
      "expirationMonth": "12",
      "expirationYear": "2026"
    }
  }
}
```

Should use transient token jti in PA-Setup request.

```
{
  "clientReferenceInformation": {
    "code": "123456789"
  },
  "tokenInformation": {
    "transientToken": "1E15TS9EFYDOI743PPTIN4EYHQT3FA7S0NE9RAKSYGBK29TWX2E654E16CC7247"
  }
}
```

Below is an example response for a setup service request

```
{
  "clientReferenceInformation": {
    "code": "9aa45838-ea02-4ba1-8066-47b55431b452"
  },
  "consumerAuthenticationInformation": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiJhbnZhiZTM2OS1kYTcxLTQwZjMtOGIwYS1hNzc1ZTUwNGRlZGQlLCJpYXQiOjE0OTk2MTYwNDksImVzcyI6IjVkZDgzYmYwMGU0MjNkMTQ0ZGRjYmFjYStSIsinW4cCI6MTY5OTYxOTY0S2wiT3JnVmVpdElkIjoiaW5jaWZyYmI3Y2UyMGQ0Y2I3ZDRkOGMyMzIyIiwiaWF0IjE5ODU0NTY0MDAwMCJ9.eyJhdGkiOiJhbnZhiZTM2OS1kYTcxLTQwZjMtOGIwYS1hNzc1ZTUwNGRlZGQlLCJpYXQiOjE0OTk2MTYwNDksImVzcyI6IjVkZDgzYmYwMGU0MjNkMTQ0ZGRjYmFjYStSIsinW4cCI6MTY5OTYxOTY0S2wiT3JnVmVpdElkIjoiaW5jaWZyYmI3Y2UyMGQ0Y2I3ZDRkOGMyMzIyIiwiaWF0IjE5ODU0NTY0MDAwMCJ9",
    "deviceDataCollectionUrl": "https://centinelapistag.cardinalcommerce.com/V1/Cruise/Collect",
    "referenceId": "699195f9-1dd6-4349-a708-5075bac57e82",
    "token": "AxizbwSTe0M7Ew8txUx5ABEBT33hvtZMAZ5QyaSZejFsgHAAeAAA4gDs"
  },
  "id": "6996160490716000504953",
```

```
"status": "COMPLETED",
"submitTimeUtc": "2023-11-10T11:34:09Z"
}
```

Field	Meaning
status	Indicates status of the call. Possible values: COMPLETED: the call was successful. FAILED: an error occurred.
consumerAuthenticationInformation.accessToken	JSON Web Token (JWT) used to authenticate the consumer with the authentication provider. Required for next step - Device Data Collection.
consumerAuthenticationInformation.deviceDataCollectionUrl	The deviceDataCollectionUrl is the location to send the Authentication JWT when invoking the Device Data collection process. Required for next step - Device Data Collection.
consumerAuthenticationInformation.referenceId	This identifier represents the device data collection session. Required for subsequent step - Payer Auth Enrollment Check.

5.2. Step 2 Device Data Collection

Device Data Collection is initiated on the front end to collect device data from the browser

5.2.1. Create Device Data Collection iframe

To capture the Device data the integrator needs to Initiate a form POST in a hidden 1 x 1 iframe to the device data collection URL. This can be achieved using the code snippet below and passing in

```
consumerAuthenticationInformation.deviceDataCollectionUrl
consumerAuthenticationInformation.accessToken
```

https://cintinelapistag.cardinalcommerce.com/V1/Cruise/Collect
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI1NjJlYTljZS00MGEwLlQzOTktYjI3OS02MmUwOz0GE0OW
I1MzU1LCJpYXQiOjE2NjA1Nzg1MjksIm1zcyI6IjVhZDZgZmYmYmU0MjNkMTQ5OGRjYmFjYSIsImV4IjoiCi01MTY2MDU0MjYyO
jE0OswiTi3JnVW5pdElkIjoianJpXzNzZmNmJmMGxZAzYTE0NjlkODZjZiwiU0MzVXJlbmNlSWQwOi01HmJmBiZyU0Zi1h
ZDQ0LlQ0NmU0T0QzZDZkMTU5NjIjF0. hKsZl1jyQZRJ2IHMgKnP-wuH0nFX90tL847AGsch9k

place the below into the `<body>` of your checkout page.

[illegible]

5.2.2. Invoke Device Data Collection Iframe

to invoke the Iframe form POST place the below Javascript outside the `</body>` element.

```
<script>
window.onload = function() {
var cardinalCollectionForm = document.querySelector('#cardinal_collection_form');
if(cardinalCollectionForm) // form exists
cardinalCollectionForm.submit();
}
</script>
```

5.2.3. Capture Result of Device Data Collection Attempt



IMPORTANT: The collection of Data device Collection must be submitted before requesting PA check Enrollment service (Step 3)

The Integrator needs to check if the Device data collection has completed successfully. This is achieved by creating an event listener and placing it after `</body>` element in the same page that sent the POST message (example below)

```
window.addEventListener("message", function(event) {
    if (event.origin === https://centinelapistag.cardinalcommerce.com) {
        console.log(event.data);
    }
}, false);
```

The event.origin URL that you use depends on whether you are in a test or production environment:

- Test: <https://centinelapistag.cardinalcommerce.com>
- Production: <https://centinelapi.cardinalcommerce.com>

Example response to the listener event

```
{
  "MessageType": "profile.completed",
  "Session Id": "f54ea591-51ac-48de-b908-eecf4ff6beff",
  "Status": true
}
```

None of the returned data needs to be stored for future use. Note that the **Session id** returned is the same as PA-Setup response field consumerAuthenticationInformation.**referenceId**

5.2.4. DDC Backup

As a backup for device data collection, if device data collection fails or timeout, it is a good idea to include the 11 device information fields listed among the optional fields for the Check Enrollment service in your request. This way, 3DS V2.x will be maintained.

Sample of device data to be included in PA-Enrollment call:

```
"consumerAuthenticationInformation": {
  "deviceChannel": "Browser"
},
"deviceInformation": {
  "ipAddress": "12.25.81.210",
  "httpAcceptBrowserValue": "Chrome",
  "httpAcceptContent": "X-Forwarded-Proto",
  "httpBrowserLanguage": "en-US",
  "httpBrowserJavaEnabled": false,
  "httpBrowserJavaScriptEnabled": true,
  "httpBrowserColorDepth": "24",
  "httpBrowserScreenHeight": "800",
  "httpBrowserScreenWidth": "1920",
  "httpBrowserTimeDifference": "300",
  "userAgentBrowserValue": "Chrome 92"
}
```

5.3. Step 3 Payer Authentication Check Enrollment



IMPORTANT: The device collection process must finish before the customer selects the option to buy. You must receive a response before requesting the Check Enrollment service otherwise the transaction will fall back to 3DS 1.0

The Check Enrollment service verifies that the card is enrolled in a card authentication (3DS 2.x)

Submit a **Payment request**, and include a Payer Auth Enrollment check field CONSUMER_AUTHENTICATION in the request to trigger a check.

5.3.1. Payer Authentication Check Enrollment request fields

	IMPORTANT: To reduce your issuer step-up authentication rates, you can send additional request data in then payment request. It is best to send all available fields.
---	---

When requesting Payment plus Check Enrollment service, you must send in the following mandatory fields

Field	Meaning
Processinginformation.actionlist	Indicates to the payment endpoint to run Check Enrollment before running payment CONSUMER_AUTHENTICATION Required
ConsumeAuthenticationInformation.returnUrl	If there is a setup in authentication the issuer needs to know where to redirect after an authentication challenge Required
ConsumeAuthenticationInformation.referenceId	Reference ID that corresponds to the device fingerprinting data that was collected previously. Returned in Figure 3.1.2 Setup Service Response Required
Payment details	Response must send either the customer's card Number or transient token, or a TMS token Required
orderInformation.billTo.firstName orderInformation.billTo.lastName orderInformation.billTo.email orderInformation.billTo.address1 orderInformation.billTo.locality orderInformation.billTo.postalCode orderInformation.billTo.country	The Check Enrolment service to determine if a Step-Up in cardholder authentication is needed, you need to supply billing address information Required

consumerAuthenticationInformation.challengeCode	This field will default to 01 on merchant configuration and can be overridden by the merchant. Need to have the value to be 04 to force authenticating the customer.
--	--

It is recommended to include as much data as possible in your request, as this will increase the chance that the Issuer will not require a challenge screen.

Please review following links for additional fields

[Payer Authentication Documentation](#)

[API Reference Assets | CyberSource Developer Center](#)

Sample PA-Enrollment check using card data

</risk/v1/authentications/>

```
{
  "clientReferenceInformation": {
    "code": "cybs_test"
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Doe",
      "address2": "Address 2",
      "address1": "1 Market St",
      "postalCode": "94105",
      "locality": "san francisco",
      "administrativeArea": "CA",
      "country": "US",
      "phoneNumber": "4158880000",
      "company": "Visa",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "10.99",
      "currency": "USD"
    }
  },
  "deviceInformation": {
    "ipAddress": "127.0.0.1",
    "httpAcceptBrowserValue": "application/json",
    "httpAcceptContent": "application/json",
    "httpBrowserLanguage": "en-US",
    "httpBrowserJavaEnabled": false,
    "httpBrowserJavaScriptEnabled": false,
    "httpBrowserColorDepth": "24",
    "httpBrowserScreenHeight": "1280",
    "httpBrowserScreenWidth": "1280",
    "httpBrowserTimeDifference": "-120",
    "userAgentBrowserValue": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/112.0.0.0 Safari/537.36 Edg/112.0.1722.48"
  },
  "buyerInformation": {
    "mobilePhone": "1245789632"
  },
  "paymentInformation": {
    "card": {
      "expirationMonth": "12",

```

```

    "expirationYear": "2027",
    "number": "4000xxxxxxx2503"
  },
  "consumerAuthenticationInformation": {
    "referenceId": "61dbf7fd-b76e-496d-973e-6d7fd0228d73", //Retrieved in PA-Setup response
    "returnUrl": https://webhook.site/,
    "deviceChannel": "Browser"
  }
}

```

Sample PA-Enrollment check using Flex Microform Transient Token

</risk/v1/authentications/>

```
{
  "clientReferenceInformation": {
    "code": "cybs_test"
  },
  "aggregatorInformation": {
    "name": "name",
    "aggregatorId": "12345",
    "subMerchant": {
      "id": "12345",
      "address1": "smart village",
      "locality": "Cairo",
      "country": "EG"
    }
  },
  "merchantInformation": {
    "merchantDescriptor": {
      "name": "ministry of ***",
      "address1": "smart village",
      "locality": "Cairo",
      "country": "EG",
      "postalCode": "100271"
    },
    "categoryCode": "644"
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Doe",
      "address2": "Address 2",
      "address1": "1 Market St",
      "postalCode": "94105",
      "locality": "san francisco",
      "administrativeArea": "CA",
      "country": "US",
      "phoneNumber": "4158880000",
      "company": "Visa",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "10.99",
      "currency": "USD"
    }
  },
  "tokenInformation": {
    "transientTokenJwt": "eyJraWwQIiwOdDBHdk5YZHUxbmtqN1ZrWFRnZ0R0WE1QWHQwOEhSbCIzImFsZyI6IjJTMjU2In0.eYkYXRhIjpb7Im51bWJlciI6IjQwMDAwMmFhYWZhYWdEOWTEiLCJ0eXB1IjoimDAxIn0sImlzcyI6IkZsZXgvMDgiLCJleHAiOjE2OTk2MTY0NjAsInR5cGUiOiJtZi0wLjExLjAiLCJpYXQiOjE2OTk2MTU1NjAsImp0aSI6IjFFMTVUUzlfV0ZZRE9JNzQzUFBUSU40RV1IUUVQzRkE3UzBORT1SQUTWUdCSzI5VFdyMkuU2NTRFMtZDQzcYNdcilCJjbm250ZW50Ijp7InBheW1lbmRJbmZvcmlhdGlvbiI6eyJjYXkiIjp7Im51bWJlciI6eyJtYXNrZWRRWYwx1ZSI6IlhyYy"
  }
}
```

```
PhYwFHyWfHWDEOTEIEILCJiaW4iI0IMDAwMDAifSwic2VjdXJpdHlwZSI6eyJ2YWx1ZSI6IjAwMSJ9fX19fQ. Fje2k5sxG2L9HavRGflyFLAo9oG7J5ES0n4Ak8yhufPrUzZHGOXPXEOTM16N_1ZF62VsuwUKrmIEkS1WB C40dn7YGs2575HYhAg9e9ad70cZEHu-F99xswujkB_9WnEmGYoLOF8bJYNJMlJJXZEJqng2aaBk00KGaXqqrUzZqw9Ne eSluPZKA15CL0Csbs4-abG1AMsnvgv_RDHU8jwuqXw98-8VXFbeeqm3JaKJMeLmb2uLB7rj4z-930F9yVwZt1PrubJ07 dWLHaF6tZ_EK7iPtCDQxzYqfshNsJ5L5LN_gS3ZW9-OEBYPu9ik29Tng-TjKDHOsYVUo1ZW73yXVabA"
```

```
    },  
    "deviceInformation": {  
      "ipAddress": "127.0.0.1",  
      "httpAcceptBrowserValue": "application/json",  
      "httpAcceptContent": "application/json",  
      "httpBrowserLanguage": "en-US",  
      "httpBrowserJavaEnabled": false,  
      "httpBrowserJavaScriptEnabled": false,  
      "httpBrowserColorDepth": "24",  
      "httpBrowserScreenHeight": "1280",  
      "httpBrowserScreenWidth": "1280",  
      "httpBrowserTimeDifference": "-120",  
      "userAgentBrowserValue": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/112.0.0.0 Safari/537.36 Edg/112.0.1722.48"  
    },  
    "buyerInformation": {  
      "mobilePhone": "1245789632"  
    },  
    "consumerAuthenticationInformation": {  
      "referenceId": "61dbf7fd-b76e-496d-973e-6d7fd0228d73", //Retrieved in PA-Setup response  
      "returnUrl": https://webhook.site/,  
      "deviceChannel": "Browser"  
    }  
}  
}
```

5.3.2. PA Enrollment Check Response data

Below is an example response for a payment plus CONSUMER_AUTHENTICATION request

Example of PENDING_AUTHENTICATION: if the customer's card is enrolled in a 3DS 2.x program and a set-up in authentication is need by the Issuer. i.e., asking the cardholder to authenticate themselves using OTP or Out of bound mobile application.

```
{
  "clientReferenceInformation": {
    "code": "3DS2 Frictionless"
  },
  "consumerAuthenticationInformation": {
    "challengeRequired": "N",
    "authenticationTransactionId": "RRUpZbg0EyAbrP4NzrT0",
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiJkNTZjZGI1Zi02YmM3LTRkOGYwOTRlbnV1ZTY4NDZjbnV5MDk1LCJpcyYxQiojE2NjZS0DU3NjUsIm1zcyI6IjVkZGZyYWYyMGU0MjNmKTM2MTc0OTM2NSwiT3JnVm5pdElkIjoiaWZgZDZmJmMGMeXzJAZyTE0NJlkODZjZiwiUGF5bG9hZCI6eyJBQ1NVcmwiOiJodHRwcovLzBtZXJjaGFudGFjc3N0YWcuY2FyZGlueWwjb21tZXJjZS5jb20vTWVyY2hhbnRBQ1NXZWlvcyJ3klqSXVNUZr3SWl3aWRHVaVF0WVRSAk5pMDBOMk50t0RCak9XSmxZalFpTENKaFkzTlVjbUz1YzBsRU1qb2lZeKEyTKRrMVlUY3RNVEpsWmkwMF1UaG1lVGd4WkRrdE5HWm10ekExtW1WallqVXlJaXdpdTJoAGJHeGxiBwRSVjsdVpHOTNMmw2WlNjNklqXQ1JbjAiLCJucmFuc2FjdGlvbkklkIjoilU1UFpiZBFeUFIclA0TnpYVDAlfSwiT2JqZW0aWZSU6F5bG9hZCI6dHJ1ZSwiUmV0dXJuVXJsIjoiaHR0cDovL2xvY2FsaG9zdDo4MDdyL3Rlc3Rpbmcmc0DA0LzEucGhwJTlwlSUyMFdQZTIwSWZyYW11JTlweUEmVG9rZW4mT3ZlcnJpZGU1MjBVUkwvd2ViL3JlY2VpcHQucGhn0.133NHv3fbuZ5CdYNU8HKoDJ23pw_s",
    "token": "AxjzbwSTZsESZt5ZFRzABIBURyHAEv9nQ6g8IW2tJMvRiy3l6ATAAAHQEW",
    "acsUrl": "https://0merchantacsstag.cardinalcommerce.com/MerchantACSWeb/creq.jsp",
    "acsReferenceNumber": "Cardinal ACS",
    "stepUpUrl": "https://centinelapistag.cardinalcommerce.com/V2/Cruise/StepUp",

```

```

    "pareq": "eyJtZXNzYwdlVHlwZSI6IkNSZXEiLCJtZXNzYwdlVmVyc2lvdjI6IjIuMS4wIiwidGhyZWVEU1NlcnZlclRyYw5zSUQiOiI1ZGVhZDRhYS1jMjQ1LTQ0YTQtYTRjNi00N2NhODBjOWJlYjQ1LCJhY3NUcmFuc01EIjo1YzA2NDk1YTctMTJlZi00YThtLTgxZDktNGZmNmZAI1MmVjYjUyIiw1Y2hhbGxlbmd1V2luZG93U2l6ZSI6IjAyIn0",
    "directoryServerTransactionId": "1e930f9e-9a33-4757-a4a6-c8bd4e1e170f",
    "veresEnrolled": "Y",
    "threeDSServerTransactionId": "5dead4aa-c245-44a4-a4c6-47ca80c9beb4",
    "acsOperatorID": "MerchantACS",
    "specificationVersion": "2.1.0",
    "acsTransactionId": "c06495a7-12ef-4a8b-81d9-4ff7052ecb52"
  },
  "errorInformation": {
    "reason": "CONSUMER_AUTHENTICATION_REQUIRED",
    "message": "The cardholder is enrolled in Payer Authentication. Please authenticate the cardholder before continuing with the transaction."
  },
  "id": "6617857651216175404953",
  "paymentInformation": {
    "card": {
      "bin": "411111",
      "type": "VISA"
    }
  },
  "status": "PENDING_AUTHENTICATION",
  "submitTimeUtc": "2025-03-06T15:09:25Z"
}

```

Key fields on a Pending authentication response, that are require in the section 3.3 Challenge URL

Field	Meaning
status	Indicates whether a challenge screen is required. Possible values: - PENDING_AUTHENTICATION: A challenge screen is required.
consumerAuthenticationInformation.stepUpUrl	The fully qualified URL that the merchant uses to post a form to the cardholder in order to complete the Consumer Authentication transaction for the Cardinal Cruise API integration.
consumerAuthenticationInformation.accessToken	JSON Web Token (JWT) used to authenticate the consumer with the authentication provider. Note - Max Length of this field is 2048 characters.
consumerAuthenticationInformation.authenticationTransactionId	Unique Identify that tracks the challenge response. That will be provide in Step 5 PA Validation
consumerAuthenticationInformation.pareq	This is used to determine the Iframe dimensions the ACS will be using. (Usage exempld in section 3.4)

EXAMPLE of Frictionless Authentication: This means the Issuer does not need a step-up in authentication.

```

{
  "statusLine": "HTTP/1.1 201 Created",
  "body": {
    "clientReferenceInformation": {
      "code": "1a955f15-64f6-4a25-afe0-c37db0245461"
    },
    "consumerAuthenticationInformation": {
      "eciRaw": "05",
      "authenticationTransactionId": "0P9XqSxjCNoV81nhg0X0",
      "strongAuthentication": {
        "OutageExemptionIndicator": "0"
      }
    }
  }
}

```

```

    "eci": "05",
    "token": "AxjzbwSTcIn60pQuHiA3ABEBURy2vRZhpAz5swNhk0ky9GLBP4wAUAAA7QXs",
    "cavv": "Y2FyZGluYWxjb21tZXJjZWZlGg=",
    "paresStatus": "Y",
    "acsReferenceNumber": "Cardinal ACS",
    "xid": "Y2FyZGluYWxjb21tZXJjZWZlGg=",
    "directoryServerTransactionId": "c8840017-2ea7-48ec-acb1-95b2b07ef00f",
    "veresEnrolled": "Y",
    "threeDSServerTransactionId": "b2cc6e11-13ae-43fe-8192-80c0f01477ca",
    "acsOperatorID": "MerchantACS",
    "ecommerceIndicator": "vbv",
    "specificationVersion": "2.1.0",
    "acsTransactionId": "43290784-70ee-4c49-a090-736fcdc3f67a"
  },
  "id": "6798344309786359504951",
  "paymentInformation": {
    "card": {
      "bin": "400000",
      "type": "VISA"
    }
  },
  "status": "AUTHENTICATION_SUCCESSFUL",
  "submitTimeUtc": "2025-03-06T12:40:31Z"
}

```

Please note that:

PA-Enrollment is considered successful frictionless authentication when..

- [Status: Authentication_Successful] AND
- [PA Transaction Status:Y] AND
- [Card Enrollment Result:Y] AND
- [3DS Server Version:2.x.0] AND
- [ECI:05 for Visa/AMEX or ECI:02 for MC]

Example of PA-Enrollment Declined Response: This means the issuer returned Cards not enrolled. In such case, you should ask the customer to try another card or to try again later.

```

{
  "clientReferenceInformation": {
    "code": "cbys_test"
  },
  "consumerAuthenticationInformation": {
    "eciRaw": "07",
    "authenticationTransactionId": "FwlsH29hnKBx8sGNdAp0",
    "strongAuthentication": {
      "OutageExemptionIndicator": "0"
    },
    "signedParesStatusReason": "01",
    "token": "AxjzbwSTa8LW4EYT+0r3ABEBURycuE7tpAfISWLuaSZeJFgn8YAcAAAAYgca",
    "cardholderMessage": "Note: Additional authentication is required by the issuing bank for this transaction, please contact bank at 123-456-7890.",
    "paresStatus": "N",
    "acsReferenceNumber": "Cardinal ACS",
    "directoryServerTransactionId": "700a0aec-05c0-4eb4-922e-5e61d8f9f5bd",
    "veresEnrolled": "Y",
    "threeDSServerTransactionId": "3e19c869-0f82-4050-a223-73e5c8c8fccf",
    "acsOperatorID": "MerchantACS",
    "specificationVersion": "2.1.0",
    "acsTransactionId": "59112902-cebd-4d60-a6d3-66871a7aa401"
  },
  "errorInformation": {
    "reason": "CONSUMER_AUTHENTICATION_FAILED",
    "message": "Encountered a Payer Authentication problem. Payer could not be authenticated."
  }
}

```

```
{
  "id": "6710207754236528904951",
  "paymentInformation": {
    "card": {
      "bin": "400000",
      "type": "VISA"
    }
  },
  "status": "DECLINED",
  "submitTimeUtc": "2022-12-14T12:26:15Z"
}
```

Response of "Declined", will mean end of transaction flow

Field	Meaning
status	- DECLINED: All the above values indicate that a challenge screen was not required, and the authorization request was submitted. - INVALID_REQUEST: An error has occurred.
consumerAuthenticationInformation.veresEnrolled	Result of the enrollment check. This field can contain one of these values: - Y: Card enrolled or can be enrolled; you must authenticate. Liability shift  IMPORTANT: StepUpUrl will be provided, unless the message contains paresStatus – due to a frictionless authentication process) - N: Card not enrolled; proceed with authorization. Liability shift. - B: Bypass authentication. No liability shift. - U: Unable to authenticate regardless of the reason. No liability shift.  INFO: This could be the DS or ACS is not available for authentication at the time of the request
consumerAuthenticationInformation.paresStatus	Raw result of the authentication check. This field can contain one of these values: - A: Proof of authentication attempt was generated. - N: Customer failed or cancelled authentication. Transaction denied. - U: Authentication not completed regardless of the reason. - Y: Customer was successfully authenticated. - R: Reject Authentication

consumerAuthenticationInformation.Eci consumerAuthenticationInformation. ecommerceIndicator	The numeric commerce indicator that indicates to the bank the degree of liability shift achieved during payer authentication processing.				
	Card scheme	ECI status	E-Commerce Indicator	pare s status	Protecte d
	All	07	Internet, vbv_failure		No liability shift
	VISA	05	vbv	Y	Liability shift
	VISA	06	vbv_attempted	A	Liability shift
	AMEX	05	aesk	Y	Liability shift
	AMEX	06	aesk_attempted	A	Liability shift
	Mastercard	01	spa_attempted	A	Liability shift
	Mastercard	02	spa	Y	Liability shift
consumerAuthenticationInformation.signedParesStatus Reason	Reason for payer authentication response status. Values' lookup here Cardinal Possible+StatusReason+values				

5.4. Step 4 Step-up Challenge Iframe

If the Status to enrollment check is a [PENDING_AUTHENTICATION](#), the integrator now must redirect the Cardholder to the Issuing bank's Access Control Service (ACS) that is responsible for validation challenge

The information required to redirect the cardholder is contained in the [PENDING_AUTHENTICATION](#) response – as shown below..

Field	Meaning	HTML
consumerAuthenticationInformation: stepUpUrl	The fully qualified URL that the merchant uses to post a form to the cardholder in order to complete the Consumer Authentication transaction for the Cardinal Cruise API integration.	Form POST Action
consumerAuthenticationInformation: accessToken	JSON Web Token (JWT) used to authenticate the consumer with the authentication provider. Note - Max Length of this field is 2048 characters.	JWT POST
consumerAuthenticationInformation. pareq	This is used to determine the Iframe dimensions the ACS will be using.	Iframe height and width

Example of a decoded **consumerAuthenticationInformation.pareq** below. The integrator can use this to determine if the issuer accepted the Iframe size sent in Payment + enrollment check via **consumerAuthenticationInformation.acsWindowSize**

```
{
  "messageType": "CReq",
  "messageVersion": "2.1.0",
  "threeDSServerTransID": "5dead4aa-c245-44a4-a4c6-47ca80c9beb4",
  "acsTransID": "c06495a7-12ef-4a8b-81d9-4ff7052ecb52",
  "challengeWindowSize": "02"
}
```

5.4.1. Create Challenge Request Iframe

Create an iframe that is the same size as the Challenge Window Size specified by decoding the JWT of **consumerAuthenticationInformation.pareq**.

MD POST Parameter: Merchant-defined data returned in the response from Issuer once Challenge is completed. This field is optional.

```
<iframe name="step-up-iframe" height="400" width="400"></iframe>
<form id="step-up-form" target="step-up-iframe" method="post" action="https://centinelapis
tag.cardinalcommerce.com/V2/Cruise/StepUp">
<input type="hidden" name="JWT" value="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiJjNDN
iMGZhMy01MGYzLTQyNGUtYjZjMi0yZj - truncated for example - UsImLzcyI6IjVkdGZyYmYwMGU0MjNkGhw
In0.VpWYp84LwvTPZWuKptFedCc0QAXMsawLg9vdzxpHcc" />
<input type="hidden" name="MD" value="optionally_include_custom_data_that_will_be_returned_
as_is"/>
</form>
```

Challenge window values and dimensions:

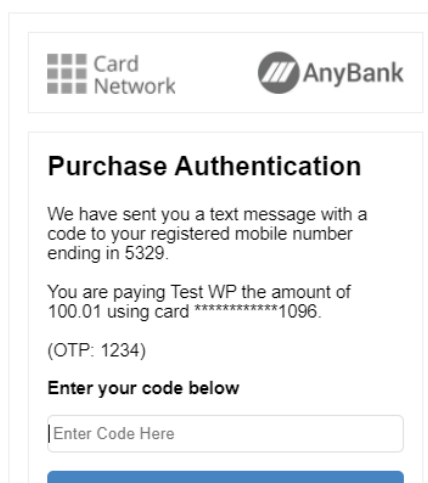
Challenge Window Size Value	Step-Up Iframe Dimensions (Width x Height)
01	250 x 400
02	390 x 400
03	500 x 600
04	600 x 400
05	Full screen

5.4.2. Invoke Challenge Request Iframe

Add the JavaScript below to invoke the iframe <form> POST. Place the JavaScript after the closing </body>. This will automatically call the POST when the page is loaded

```
<script>
  window.onload = function() {
    var stepUpForm = document.querySelector('#step-up-form');
    if(stepUpForm) // Step-Up form exists
      stepUpForm.submit();
  }
</script>
```

5.4.3. Capture Result of Challenge Attempt



After the customer interacts with the issuing bank, the user is redirected back to the **consumerAuthenticationInformation.returnUrl** within the Iframe as specified in Section 3.3.1

Example response of the data returned after completing step-up/challenge

```
{
  "TransactionId"="RRuPZbg0EyAbrP4NzrT0"
  "MD"="optionally_include_custom_data_that_will_be_returned_as_is"
}
```

None of the returned data needs to be stored for future use. Note that the **Transaction id** returned is the same as the **consumerAuthenticationInformation.authenticationTransactionId** that is used in Step 5 Validation

5.5. Payer Authentication Validation

After the Step-up/Challenge is completed, the integrator will need to submit a validation call to pull back the results of authentication as well as the authentication values required to be passed in authorization

5.5.1. Payer Authentication Validation Request

Field	Meaning
-------	---------

Payment details	send either the customer's card Number or transient token, or a TMS token Optional
ConsumeAuthenticationInformation.authenticationTransactionId	Requested to validate authentication. Value is communicated to merchant return URL "Webhook". Value also included in PA-Enrollment response. Required

Please review these links for additional fields

[Payer Authentication Documentation](#) and [API Reference Assets | CyberSource Developer Center](#)

PA-Validate using card data

</risk/v1/authentication-results/>

```
{
  "clientReferenceInformation": {
    "code": "cybs_test"
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Doe",
      "address2": "Address 2",
      "address1": "1 Market St",
      "postalCode": "94105",
      "locality": "san francisco",
      "administrativeArea": "CA",
      "country": "US",
      "phoneNumber": "4158880000",
      "company": "Visa",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "10.99",
      "currency": "USD"
    }
  },
  "buyerInformation": {
    "mobilePhone": "1245789632"
  },
  "paymentInformation": {
    "card": {
      "expirationMonth": "12",
      "expirationYear": "2025",
      "number": "4000xxxxxxx1091",
      "type": "001" //to grant correct indicator value in PA-Validate response
    }
  },
  "consumerAuthenticationInformation": {
    "authenticationTransactionId": "rTjeCT3j1LXck2AQWmi0",
    "transactionMode": "S"
  }
}
```

[/risk/v1/authentication-results/](#)

5.5.2. PA-Validate Response data

Field	Meaning
status	Indicates status of the transaction. Possible values:
consumerAuthenticationInformation.authenticationResult	Raw authentication data that comes from the card-issuing bank. Primary authentication field that indicates if authentication was successful and if liability shift occurred. Possible values: -1: Invalid PARES. -0: Successful validation. -1: Cardholder is not participating, but the attempt to authenticate was recorded. -6: Issuer unable to perform authentication. -9: Cardholder did not complete authentication.
consumerAuthenticationInformation.paresStatus	Raw result of the authentication check. This field can contain one of these values: - A: Proof of authentication attempt was generated. - N: Customer failed or cancelled authentication. Transaction denied. - U: Authentication not completed regardless of the reason. - Y: Customer was successfully authenticated.

Field	Meaning																																								
consumerAuthenticationInformation. ECI consumerAuthenticationInformation. Indicator	The numeric commerce indicator that indicates to the bank the degree of liability shift achieved during payer authentication processing. <table><tr><th>Card scheme</th><th>ECI status</th><th>E-Commerce Indicator</th><th>pares status</th><th>Protected</th></tr><tr><td>All</td><td>07</td><td>Internet, vbv_failure</td><td></td><td>No liability shift</td></tr><tr><td>VISA</td><td>05</td><td>vbv</td><td>Y</td><td>Liability shift</td></tr><tr><td>VISA</td><td>06</td><td>vbv_attempted</td><td>A</td><td>Liability shift</td></tr><tr><td>AMEX</td><td>05</td><td>aesk</td><td>Y</td><td>Liability shift</td></tr><tr><td>AMEX</td><td>06</td><td>aesk_attempted</td><td>A</td><td>Liability shift</td></tr><tr><td>Mastercard</td><td>01</td><td>spa_attempted</td><td>A</td><td>Liability shift</td></tr><tr><td>Mastercard</td><td>02</td><td>spa</td><td>Y</td><td>Liability shift</td></tr></table>	Card scheme	ECI status	E-Commerce Indicator	pares status	Protected	All	07	Internet, vbv_failure		No liability shift	VISA	05	vbv	Y	Liability shift	VISA	06	vbv_attempted	A	Liability shift	AMEX	05	aesk	Y	Liability shift	AMEX	06	aesk_attempted	A	Liability shift	Mastercard	01	spa_attempted	A	Liability shift	Mastercard	02	spa	Y	Liability shift
Card scheme	ECI status	E-Commerce Indicator	pares status	Protected																																					
All	07	Internet, vbv_failure		No liability shift																																					
VISA	05	vbv	Y	Liability shift																																					
VISA	06	vbv_attempted	A	Liability shift																																					
AMEX	05	aesk	Y	Liability shift																																					
AMEX	06	aesk_attempted	A	Liability shift																																					
Mastercard	01	spa_attempted	A	Liability shift																																					
Mastercard	02	spa	Y	Liability shift																																					

Example of successful PA-Validate response:

```
{
  "clientReferenceInformation": {
    "code": "1673333815476"
  },
  "consumerAuthenticationInformation": {
    "indicator": "vbv",
    "eciRaw": "05",
    "authenticationResult": "0",
    "strongAuthentication": {
      "OutageExemptionIndicator": "0"
    },
    "authenticationStatusMsg": "Success",
    "eci": "05",
    "token": "AxjzbwSTbQPWkDb3BH+5ABEBURyn8G3vpAr6C8GGTSTL0YsE/jADwAAATBJS",
    "cavv": "MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=",
    "paresStatus": "Y",
    "xid": "MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=",
    "directoryServerTransactionId": "45929715-c174-4999-93f0-26f313208577",
    "threeDSServerTransactionId": "45f6bac4-1e48-4502-ab38-362638762aa5",
    "specificationVersion": "2.1.0",
    "acsTransactionId": "862c2a7f-85fa-4a8f-9422-897115841e44"
  },
  "id": "6733338153896863104953",
  "paymentInformation": {
    "card": {
      "bin": "400000",
      "type": "VISA"
    }
  },
  "status": "AUTHENTICATION_SUCCESSFUL",
  "submitTimeUtc": "2023-01-10T06:56:55Z"
}
```

Example of failed PA-Validate response:

```
{
  "clientReferenceInformation": {
    "code": "1673280584530"
  },
  "consumerAuthenticationInformation": {
    "indicator": "internet",
    "eciRaw": "07",
    "paresStatus": "U",
    "authenticationResult": "6",
    "directoryServerTransactionId": "1641c3e4-49de-4a0d-bdb6-e6b2d02d12d2",
    "strongAuthentication": {
      "OutageExemptionIndicator": "0"
    },
    "authenticationStatusMsg": "Issuer unable to perform authentication",
  },
}
```

```
{
  "threeDSSTransactionId": "ae2e1106-6407-4b16-b026-50655951e4d0",
  "eci": "07",
  "specificationVersion": "2.1.0",
  "token": "AxjzbwSTbPxza3ZnZgIZABEBURyn1rXPPAr6C8GGTSTL0YsE/jADwAAASBJE",
  "acsTransactionId": "3be39363-ea71-49d2-9c07-036447f131f5"
},
{
  "id": "6732805844306441404953",
  "paymentInformation": {
    "card": {
      "bin": "400000",
      "type": "VISA"
    }
  }
},
{
  "status": "AUTHENTICATION_SUCCESSFUL",
  "submitTimeUtc": "2023-01-09T16:09:44Z"
}
}
```

could use the value of **signedParesStatusReason** to differentiate between PA status reasons. The lookup of the expected values for **signedParesStatusReason** here [EMV 3DS Test Cases](#)

You will use ECI to choose to proceed with authorization or not and should include the following in authorization request to grant liability shift.

Visa Card Sample

```
{
  "processingInformation": {
    "commerceIndicator": "vbv" //mandatory
  },
  "consumerAuthenticationInformation": {
    "cavv": "MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=", //mandatory
    "paresStatus": "Y",
    "xid": "MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=", //mandatory if returned from PA
    "directoryServerTransactionId": "c4085dad-247e-4a8d-811c-f5590091a53c",
    "veresEnrolled": "Y",
    "paSpecificationVersion": "2.1.0"
  }
}
```

Mastercard Sample

```
{
  "processingInformation": {
    "commerceIndicator": "spa" //mandatory
  },
  "consumerAuthenticationInformation": {
    "ucafAuthenticationData": "Y2FyZGluYXxjb21tZXJjZW1dGg=", //mandatory
    "paresStatus": "Y",
    "ucafCollectionIndicator": "2", //mandatory
    "directoryServerTransactionId": "e372b726-5c52-4b70-b63c-7d6c099907ba",
    "veresEnrolled": "Y",
    "paSpecificationVersion": "2.1.0"
  }
}
```

PA-Validate response for Mastercard is different than the other schemes.

For MasterCard only, **consumerAuthenticationInformation.ucafCollectionIndicator** is the numeric electronic commerce indicator (ECI) and this field is absent when authentication fails.

You must send this value to your payment processor in the request for card authorization.

This field contains one of these values:

- 0: UCAF collection is not supported at your website. **Authentication was not completed.**
- 1: UCAF collection is supported at your website, and UCAF was populated. **Authentication was not completed.**

- 2: UCAF collection is supported at your website, and UCAF was populated. **Customer completed authentication.**

consumerAuthenticationInformation.ucafAuthenticationData “AAV” is a unique identifier generated by the card-issuing bank for Mastercard Identity Check transactions after the customer is authenticated. You must include this data in the authorization request.

		IMPORTANT: It's highly recommended to use a unique Merchant Reference Number “MRN” per flow.					
Date	Request ID	Merchant Reference Number	Amount	Currency	Applications	Payment Method	Commerce
Press Enter to filter results		Press Enter to filter results	Press Enter	Press E	Filter		
Oct 01 2024 01:28:51 PM EET	7277821311616449704953	790ba0af-c83e-48ba-8b02-fac450824596	13.00	USD	- Success Payer Authentication Validation - Success Subscription Creation - Success Card Payments - Settlement - Success Decision Manager - Success Card Payments - Authorization	Visa	vbv
Oct 01 2024 01:27:51 PM EET	727782078156543004951	790ba0af-c83e-48ba-8b02-fac450824596			- Not Run Card Payments - Settlement - Not Run Card Payments - Authorization - Challenge Payer Authentication Enrollment - Not Run Subscription Creation	Visa	
Oct 01 2024 01:25:59 PM EET	7277819592846429204953	790ba0af-c83e-48ba-8b02-fac450824596			Success Payer Authentication Setup	Visa	

5.5.3. PA-Validate Scenarios

Case	eciRaw	paresStatus	cavv	authenticationResult	authenticationStatusMsg	status	indicator
Successful authentication	Visa: 05 MC: 02	Y	#value	0	Success	AUTHENTICATION_SUCCESSFUL	Visa: vbv MC: spa AMEX: aesk
Issuer is not supporting 3DS	Visa: 06 MC: 01	A	#value	0	Success	AUTHENTICATION_SUCCESSFUL	vbv_attempted spa_attempted
Failed cardholder challenge	Visa: 07 MC: 00	N	No value	9	User failed authentication Payer cannot be authenticated	AUTHENTICATION_FAILED	internet OR vbv_failure
Unavailable authentication Customer hasn't finalized Authentication yet	Visa: 07 MC: 00	U	No value	6	Issuer unable to perform authentication	AUTHENTICATION_SUCCESSFUL	internet OR vbv_failure

	IMPORTANT: It's recommended to have a double defense for PA by DM Post-Auth rule or by PaRC rule to confirm PA done properly
--	---

6. Payments

Payment REST API could perform multiple functions besides Authorization such as PA-Enrollment check, PA-Validate, DM, TMS token creation, and payment settlement (Capture).

When requesting Payment, you must send in the following mandatory fields

Field	Meaning
Payment details	Response must send either the customer's card Number or transient token, or a TMS token Required
orderInformation.billTo.firstName orderInformation.billTo.lastName orderInformation.billTo.email orderInformation.billTo.address1 orderInformation.billTo.locality orderInformation.billTo.postalCode orderInformation.billTo.country	The Check Enrolment service to determine if a Step-Up in cardholder authentication is needed, you need to supply billing address information Required

Please note that, there is a feature "Relaxed AVS" that makes bill to address fields optional in the authorization request. Check here for more details [Relaxed AVS](#)

6.1. Authorization + Capture + TMS/NT

Should include PA result to grant liability shift

Sample Authorization + Capture combined request using Visa card:
</pts/v2/payments/>

```
{
  "clientReferenceInformation" : {
    "code" : "3DS2"
  },
  "processingInformation" : {
    "authorizationOptions": {
      "capture": true,           //to combine capture with authorization
      "commerceIndicator": "vbv", // (MOTO or internet for no PA transactions)
    },
    "orderInformation" : {
      "billTo" : {
        "firstName": "John",
        "lastName": "Doe",
        "address1": "1 Market St",
        "address2": "metro 3",
        "locality": "san francisco",
        "administrativeArea": "CA",
        "postalCode": "94105",
        "country": "US",
        "email": "test@cybs.com",
        "phoneNumber": "4158880000"
      }
    }
  }
}
```

```

    },
    "amountDetails" : {
      "totalAmount" : "103.00",
      "currency" : "USD"
    }
  },
  "paymentInformation" : {
    "card" : {
      "expirationYear" : "2027",
      "number" : "4456xxxxxxxx2503",
      "securityCode" : "xxx",
      "expirationMonth" : "12",
      "type" : "001"
    }
  },
  "consumerAuthenticationInformation": { //Doesn't exist in MOTO transaction
    "cavv": "MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=", //mandatory for PA liability shift
    "paresStatus": "Y",
    "xid": "MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=", //mandatory for PA liability shift
    "directoryServerTransactionId": "dc1350ac-2b8c-4246-be89-5bcf32726f21",
    "veresEnrolled": "Y",
    "paSpecificationVersion": "2.1.0"
  },
  "deviceInformation": { //Device FingerPrint info in case of combined call for DM
    "ipAddress": "71.56.23.21",
    "fingerprintSessionId": "hlh135dddd235"
  },
  "merchantDefinedInformation": [
    {
      "key": "1",
      "value": "test value 1"
    },
    {
      "key": "2",
      "value": "test value 2"
    },
    {
      "key": "3",
      "value": "test value 3"
    }
  ]
}

```

Sample Authorization + Capture combined request using Mastercard PAN:

</pts/v2/payments/>

```

{
  "clientReferenceInformation" : {
    "code" : "3DS2"
  },
  "processingInformation" : {
    "authorizationOptions": {
      "capture": true, //to combine capture with authorization
      "commerceIndicator": "spa", //(MOTO or internet for no PA transactions)
    },
    "orderInformation" : {
      "billTo" : {
        "firstName": "John",
        "lastName": "Doe",
        "address1": "1 Market St",
        "address2": "metro 3",
        "locality": "san francisco",
        "administrativeArea": "CA",

```

```

        "postalCode": "94105",
        "country": "US",
        "email": "test@cybs.com",
        "phoneNumber": "4158880000"
    },
    "amountDetails" : {
        "totalAmount" : "103.00",
        "currency" : "USD"
    }
},
"paymentInformation" : {
    "card" : {
        "expirationYear" : "2027",
        "number" : "5200xxxxxxxx2151",
        "securityCode" : "xxx",
        "expirationMonth" : "12",
        "type" : "002"
    }
},
"consumerAuthenticationInformation": { //Doesn't exist in MOTO transaction
    "ucafAuthenticationData": "Y2FyZGluYXxb21tZXJjZWZlZGg=", //mandatory
    "paresStatus": "Y",
    "ucafCollectionIndicator": "2", //mandatory
    "directoryServerTransactionId": "e372b726-5c52-4b70-b63c-7d6c099907ba",
    "veresEnrolled": "Y",
    "paSpecificationVersion": "2.1.0"
},
"deviceInformation": { //Device FingerPrint info in case of combined call for DM
    "ipAddress": "71.56.23.21",
    "fingerprintSessionId": "hlh135dddd235"
},
"merchantDefinedInformation": [
    {
        "key": "1",
        "value": "test value 1"
    },
    {
        "key": "2",
        "value": "test value 2"
    },
    {
        "key": "3",
        "value": "test value 3"
    }
]
}

```

Sample Authorization + Capture combined request using Transient Token: </pts/v2/payments/>

```

{
  "clientReferenceInformation" : {
    "code" : "3DS2"
  },
  "aggregatorInformation": {
    "name": "name",
    "aggregatorId": "12345",
    "subMerchant": {
      "id": "12345",
      "address1": "smart village",
      "locality": "Cairo",
      "country": "EG"
    }
  }
}

```

```

},
"merchantInformation": {
  "merchantDescriptor": {
    "name": "ministry of **",
    "address1": "smart village",
    "locality": "Cairo",
    "country": "EG",
    "postalCode": "100271"
  },
  "categoryCode": "644"
},
"processingInformation": {
  "authorizationOptions": {
    "capture": true, //to combine capture with authorization
    "commerceIndicator": "vbv", // (MOTO or internet for no PA transactions)
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Doe",
      "address1": "1 Market St",
      "address2": "metro 3",
      "locality": "san francisco",
      "administrativeArea": "CA",
      "postalCode": "94105",
      "country": "US",
      "email": "test@cybs.com",
      "phoneNumber": "4158880000"
    },
    "amountDetails": {
      "totalAmount": "103.00",
      "currency": "USD"
    }
  },
  "tokenInformation": {
    "transientTokenJwt": "eyJraWQiOiIwODBHdk5YZHUxbmtqN1ZrWFRnZ0R0WE1QWHQwOEhSbCIzImFsZyI6IjJmU2InOyJkYXRhIjp7Im51bWJlcii6IjQwMDAwMmFhYWFhYWdEwOTEiLCJ0eXB1IjoimDAXIn0sImIzcyI6IkZsZXgvdG9uZlFVb0ZRE9JNzQzUFBUSU40RVlIUUVQzRkE3UzBORTlSQUtTWUdCSzI5VFdYMkU2NTRFMtZDQzcyNDciLCJjb250ZW50Ijp7InBheW1lbnRjb2Vzcm1hdGlvbiI6eyJYXJkIjp7Im51bWJlcii6eyJtYXNrZWRYWx1ZSI6IjYwMSJ9fXJ9fQ.Fje2k5sxG2L9HavRGfLyFLAo9oG7J5ES0n4Ak8yhufPrUzZHG0PXEOTM16N_1Zf62VsuwUKrmIEkS1WB C40dn7YGs2575HYhAg9e9ad70cZEHu-F99xswujkB_9WnEmGYoLOF8bJYNjMIJXZEJqng2aaBk00KGaXqqrUzZqW9Ne eSluPZKA15CL0Csbs4-abG1AMsngv_RDHU8jwuqXw98-8VXFbeeQM3JaKJMeLmB2uLB7rj4z-930F9yVWZt1PrubJ07 dWlHaf6tZ_EK7iPtCDQxZyqfshNsJ5L5LN_gS3Zw9-OEBYPu9ik29Tng-TjKDHOSoyVUo1ZW73yXVabA"
  },
  "consumerAuthenticationInformation": {
    "cavv": "MTIzNDU2Nzg5MDYmZQ1Njc4OTA=", //mandatory for PA liability shift
    "paresStatus": "Y",
    "xid": "MTIzNDU2Nzg5MDYmZQ1Njc4OTA=", //mandatory for PA liability shift
    "directoryServerTransactionId": "dc1350ac-2b8c-4246-be89-5bcf32726f21",
    "veresEnrolled": "Y",
    "paSpecificationVersion": "2.1.0"
  },
  "merchantDefinedInformation": [
    {
      "key": "1",
      "value": "test value 1"
    },
    {
      "key": "2",
      "value": "test value 2"
    },
    {
      "key": "3",
      "value": "test value 3"
    }
  ]
}

```

```
}
]
}
```

Sample Authorization response:

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "consumerAuthenticationInformation": {
    "token": "Axj/7wSTcN4D3WhGyGkXABEg3YsXDBixayo1WCwa1oKiOW9/1+wFRHLe/6/ekC5imCkMmkmXoxYJ/
GQHSbhvAe60I2Q0i4AAnyPu"
  },
  "id": "6804399692866399004951",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "12.21",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "scheme": "VISA CREDIT",
    "bin": "400000",
    "accountType": "Visa Traditional",
    "issuer": "INTL HDQTRS-CENTER OWNED",
    "card": {
      "type": "001"
    },
    "binCountry": "US"
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "cardVerification": {
      "resultCode": ""
    },
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "71180115JFUA05VA",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2023-04-02T12:52:49Z"
}
```

```
}

```

6.2. Card Verification Using Zero Authorization

Merchant could perform Payer Authentication and Authorization using zero amount to verify the card and to tokenize it for future usage and Cybersource will convert this Auth request into an account verification request.

For card verification PA-Enrollment call, use

consumerAuthenticationInformation.strongAuthentication.authenticationIndicator = 03

6.3. Separate Capture for Payments

A capture is a follow-on transaction to an authorization. It is used to transfer the authorized funds from the customer's account to the merchant account.

To link the Authorization transaction to the capture transaction, you include a request ID in your capture request.

Captures are typically not performed in real time. They are placed in a batch file and sent to the processor and the processor settles all of the captures at one time.

It is important to capture funds in a timely manner. The issuing bank sets the expiration time interval but most authorizations expire within five to seven days.

To capture an authorization that expired with the issuing bank, you can resubmit the authorization request.

Example: Capturing an Authorization

[/pts/v2/payments/{id}/captures](#)

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3" // should be same as authorization MRN
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "102.21",
      "currency": "USD"
    }
  }
}
```

Response:

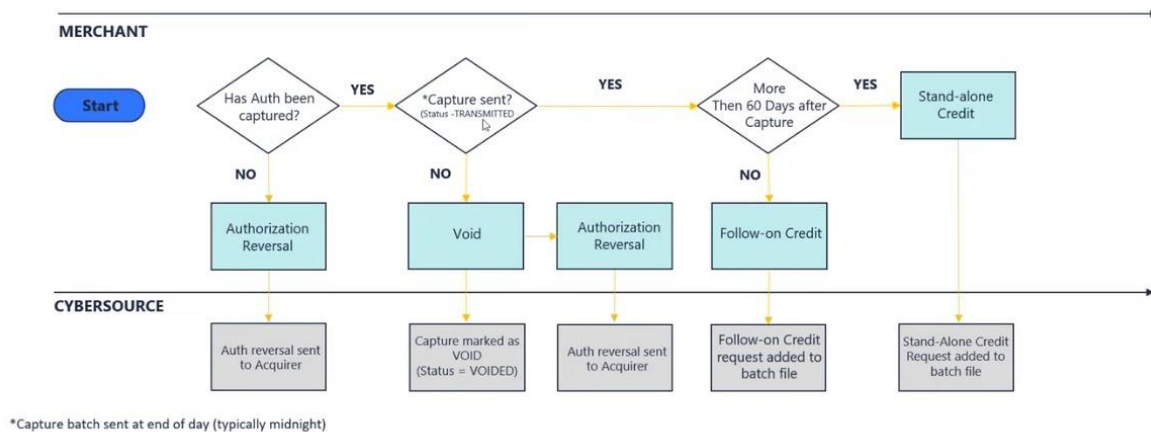
```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/captures/6920818275926471904953/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/captures/6920818275926471904953"
    }
  }
},
```

```
"clientReferenceInformation": {  
  "code": "TC50171_3"  
},  
"id": "6920818275926471904953",  
"orderInformation": {  
  "amountDetails": {  
    "totalAmount": "102.21",  
    "currency": "USD"  
  }  
},  
"reconciliationId": "703647180L6IUOMW",  
"status": "PENDING",  
"submitTimeUtc": "2023-08-15T06:43:47Z"  
}
```

7. Reverse Payments

Payment transaction could be reversed in many ways depending on settlement status and original transaction method.

Diagram below presents order cancellation workflow on the different stages of the order



7.1. Authorization Reversal

Case: To reverse the authorizations with no Capture

API: The backend to be called in case of authorization Reversal is </pts/v2/payments/{id}/reversals> where {id} is the authorized transaction RID.

Authorization Reversal example for the RID **6712818764556355104951**

URL: </pts/v2/payments/6712818764556355104951/reversals>

Request:

```
{
  "clientReferenceInformation": {
    "code": "TC50171_689"
  },
  "reversalInformation": {
    "amountDetails": {
      "totalAmount": "102.21"
    },
    "reason": "testing"
  }
}
```

Response:

```
{
  "_links": {
    "self": {
      "method": "GET",
      "href": "/pts/v2/reversals/6716119170516336204951"
    }
  },
  "clientReferenceInformation": {
```

```
{
  "code": "TC50171_689",
},
"id": "6716119170516336204951",
"orderInformation": {
  "amountDetails": {
    "currency": "USD"
  }
},
},
"processorInformation": {
  "responseCode": "100"
},
},
"reversalAmountDetails": {
  "reversedAmount": "102.21",
  "currency": "USD"
},
},
"status": "REVERSED",
"submitTimeUtc": "2022-12-21T08:38:37Z"
}
```

Transaction Management / [Transactions](#)

Transaction Details

Request ID: 6716119170516336204951

Order Information		Transaction History			
Merchant ID: 25071980	Request ID: 6716119170516336204951	Type	Request ID	Amount	Date & Time
Merchant Reference Number: TC50171_689	Date / Time: Dec 21 2022 10:38:37 AM EET	Credit Card Authorization	6712818764556355104951	102.21 USD	Dec 17 2022 02:57:56 PM EET
Email Address: test@cybs.com	Account Details: #1111	Credit Card Full Authorization Reversal	6716119170516336204951	102.21 USD	Dec 21 2022 10:38:37 AM EET
WHOIS Search					

Customer Information		Request Information		Error Information
Billing Information		Client App: REST API	Client Application Version: 1.0	No Data Available.
First Name: JOHN	Last Name: DOE	Reason Code: 100 Code Explanation	Reply Message: Request was processed successfully.	
Address:	City:			

7.2. Void a Payment

Case: to reverse Authorization + Capture (Pending status)

API: The backend to be called to void a payment is </pts/v2/payments/{id}/voids> where {id} is the authorized transaction RID.

Request ID: 6716129382086305504953

Order Information		Transaction History				
Merchant ID: 25071980	Request ID: 6716129382086305504953	Type	Request ID	Amount	Date & Time	Status
Merchant Reference Number: TC50171_3	Date / Time: Dec 21 2022 10:55:38 AM EET	Credit Card Authorization	6716129382086305504953	10.21 USD	Dec 21 2022 10:55:38 AM EET	100
		Credit Card Settlement	6716129382086305504953	10.21 USD	Dec 21 2022 10:55:38 AM EET	PENDING

Void example for the RID **6716129382086305504953**

URL: </pts/v2/payments/6716129382086305504953/voids>

Request:

```
{
  "clientReferenceInformation": {
    "code": "test_void"
  }
}
```

```
}
```

Response:

```
{
  "_links": {
    "self": {
      "method": "GET",
      "href": "/pts/v2/voids/6716136885746682904951"
    }
  },
  "clientReferenceInformation": {
    "code": "test_void"
  },
  "id": "6716136885746682904951",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "status": "VOIDED",
  "submitTimeUtc": "2022-12-21T09:08:09Z",
  "voidAmountDetails": {
    "currency": "USD",
    "voidAmount": "10.21"
  }
}
```

Void transaction

Request ID: 6716136887290177135870

Order Information		Transaction History	
Merchant ID: 25071980	Request ID: 6716136887290177135870	Related Transactions	
Merchant Reference Number: test_void	Date / Time: Dec 21 2022 11:08:08 AM EET	Type	Request ID
Email Address: test@cybs.com WHOIS Search	Account Details:	Credit Card Authorization	6716129382086305504953
		Payment Cancel	6716136885746682904951
		Voided Transactions	6716136887290177135870

Original authorization transaction

Request ID: 6716129382086305504953

Order Information		Transaction History				
Merchant ID: 25071980	Request ID: 6716129382086305504953	Type	Request ID	Amount	Date & Time	Status
Merchant Reference Number: TC50171_3	Date / Time: Dec 21 2022 10:55:38 AM EET	Credit Card Authorization	6716129382086305504953	10.21 USD	Dec 21 2022 10:55:38 AM EET	100
Email Address: test@cybs.com WHOIS Search	Account Details: #1111	Credit Card Settlement	6716129382086305504953	10.21 USD	Dec 21 2022 10:55:38 AM EET	VOIDED
		Related Transactions				
		Type	Request ID	Amount	Date & Time	Status
		Payment Cancel	6716136885746682904951		Dec 21 2022 11:08:08 AM EET	
		Voided Transactions	6716136887290177135870		Dec 21 2022 11:08:08 AM EET	

7.3. VOID a CaptureCase: to reverse Capture done separately with (Pending status)

API: The backend to be called to void a payment is </pts/v2/captures/{id}/voids> where {id} is the capture transaction RID.

Capture VOID example for RID **6944142368656459604953**

URL: </pts/v2/captures/6944142368656459604953/voids>

Request:

```
{
  "clientReferenceInformation": {
    "code": "1694414107411"
  }
}
```

Response:

```
{
  "_links": {
    "self": {
      "method": "GET",
      "href": "/pts/v2/voids/6944142368656459604953"
    }
  },
  "clientReferenceInformation": {
    "code": "1694414107411"
  },
  "id": "6944142368656459604953",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "status": "VOIDED",
  "submitTimeUtc": "2023-09-11T06:53:23Z",
  "voidAmountDetails": {
    "currency": "usd",
    "voidAmount": "10.00"
  }
}
```

Transaction History

Type	Request ID	Amount	Date & Time	Status	Reason
Card Authorization	6944141549536431904001	10.00 USD	Sep 11 2023 08:35:54 AM EET		100
Card Settlement	6944142368656459604953	10.00 USD	Sep 11 2023 08:37:16 AM EET	VOIDED	100

Related Transactions

Type	Request ID	Amount	Date & Time	Status	Reason
Voided Transactions	6944142958156486704953		Sep 11 2023 08:38:15 AM EET		

7.4. Refund a Payment

Case: Authorization + Capture (Transmitted status)

API: The backend to be called to refund a payment is </pts/v2/payments/{id}/refunds> where {id} is the authorized transaction RID.

Refund example for the RID 6695543348496530604001

URL: </pts/v2/payments/6695543348496530604001/refunds>

Request ID: 6695543348496530604001

Order Information		Transaction History				
Merchant ID: 25071980	Request ID: 6695543348496530604001	Type	Request ID	Amount	Date & Time	Status
Merchant Reference Number: 1669554334829	Date / Time: Nov 27 2022 03:05:34 PM EET	Credit Card Authorization	6695543348496530604001	10.00 USD	Nov 27 2022 03:05:34 PM EET	
		Credit Card Settlement	6695543348496530604001	10.00 USD	Nov 27 2022 03:05:34 PM EET	TRANSMITTED

Request:

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "10.00",
      "currency": "USD"
    }
  }
}
```

Response:

```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/refunds/6716167826826201804951/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/refunds/6716167826826201804951"
    }
  },
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "id": "6716167826826201804951",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "processorInformation": {
    "approvalCode": "888888",
    "responseCode": "100"
  },
  "reconciliationId": "75392241IBDGMFX6",
  "refundAmountDetails": {
    "currency": "USD",
    "refundAmount": "10.00"
  },
  "status": "PENDING",
  "submitTimeUtc": "2022-12-21T09:59:43Z"
}
```

Request ID: 6716167826826201804951

Order Information		Transaction History			
Merchant ID: 25071980	Request ID: 6716167826826201804951	Type	Request ID	Amount	Date & Time
Merchant Reference Number: TC50171_3	Date / Time: Dec 21 2022 11:59:42 AM EET	Credit Card Authorization	6695543348496530604001	10.00 USD	Nov 27 2022 03:05:34 PM EET
Email Address: null@cybersource.com WHOIS Search	Account Details: #1111	Credit Card Settlement	6695543348496530604001	10.00 USD	Nov 27 2022 03:05:34 PM EET TRANSMITTED
		Credit Card Credit Authorization	6716167826826201804951	10.00 USD	Dec 21 2022 11:59:42 AM EET
		Credit Card Credit	6716167826826201804951	10.00 USD	Dec 21 2022 11:59:42 AM EET PENDING

7.5. Refund Capture

Case: Capture (Transmitted status)

API: The backend to be called to refund a payment is [/pts/v2/captures/{id}/refunds](#) where {id} is the Capture transaction RID.

Refund Capture example for the RID 7397975103316263604805

POST: [/pts/v2/captures/7397975103316263604805/refunds](#)

Request:

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "60.00",
      "currency": "USD"
    }
  }
}
```

Response:

```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/refunds/7399497853186177804807/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/refunds/7399497853186177804807"
    }
  },
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "id": "7399497853186177804807",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "processorInformation": {
    "approvalCode": "888888",
    "responseCode": "100"
  }
}
```

```

"reconciliationId": "73164297L733POVB",
"refundAmountDetails": {
  "currency": "USD",
  "refundAmount": "60.00"
},
"status": "PENDING",
"submitTimeUtc": "2025-02-19T07:23:05Z"
}

```

Type	Request ID	Amount	Date & Time	Status	Reason
Card Authorization	7397974031426164704807	100.00 USD	Feb 17 2025 03:03:23 PM EET		100
Card Settlement	7397975103316263604805	60.00 USD	Feb 17 2025 03:05:10 PM EET	TRANSMITTED	100
Credit Card Credit Authorization	7399497853186177804807	60.00 USD	Feb 19 2025 09:23:05 AM EET		
Card Credit	7399497853186177804807	60.00 USD	Feb 19 2025 09:23:05 AM EET	PENDING	100

7.6. Bulk Transactions

Could perform bulk Reverse, VOID, and Refunds using EBC Batch Transaction Upload tool.

The screenshot displays the Cybersource Virtual Terminal interface for batch transaction uploads. The top navigation bar includes links for Feedback, Support, Notifications, and the user profile 'ahsan'. The left sidebar lists various tools, with 'Batch Transaction Upload' highlighted. The main content area shows the 'Batch Transaction Upload' page, which includes a search bar, an 'Add filter' button, and a section for 'Applied filters' (currently empty). A right sidebar lists transaction types: Authorizations, Sales, Captures, Credits, Electronic Check Debits, and Electronic Check Credits. The user is logged in as 'ahsan'.

Download the template, fill it, and upload it.

Some templates are not included in the ready list so, you will need to get them from support.

Check this article for details [Bulk authorization reversal](#)

You can also make actions on multiple transactions like Reverse and VOID by using EBC.

The screenshot shows the Cybersource Transaction Management interface. The left sidebar contains navigation links: Dashboard, Invoicing, Virtual Terminal, Transaction Management (expanded), Transactions (with 'Transaction by Phase' highlighted), Secure Acceptance, Decision Manager, Token Management, Recurring Billing, and Device Management. The main content area shows 'Account ID ahsamy_acct | Merchant ID ahsamy' and 'Transaction Management Transactions by Phase'. A search bar is present with filters for 'Phase *', 'Date', and 'Application'. The 'Phase *' dropdown menu is open, showing options: 'Settlements Pending Batch', 'Authorizations Ready to Settle', 'Exception Search', and 'Authorizations Needing Review'. The 'Date' and 'Application' filters are both set to 'Unused'.

For example, to VOID multiple transactions,

- Select Transaction Management > Transactions by Phase.
- From Phase: No Phase Selected, select Settlements Pending Batch.
- Transactions that have not been batched appear below in the results section.
- Select the box to the left of each transaction you wish to void.
- Select VOID from the top right.

Details in this article [How to Void Multiple Transactions](#)

8. Transaction Search

8.1. Search using query

Could use the backend [/tss/v2/searches](#) to search transactions using query inputs.

It's helpful to search using merchant reference number to get transaction details in case of timeouts

Sample search request using merchant reference number TC50171_689

POST [/tss/v2/searches](#)

```
{
  "save": "false",
  "name": "MRN",
  "timezone": "America/Chicago",
  "query": "clientReferenceInformation.code:TC50171_689 AND submitTimeUtc:[NOW/DAY-7DAYS TO NOW/DAY+1D
AY] AND orderInformation.amountDetails.totalAmount:102.21",
  "offset": 0,
  "limit": 100,
  "sort": "id:asc,submitTimeUtc:asc"
}
```

Search output:

```
{
  "_links": {
    "self": {
      "href": "https://apitest.cybersource.com/tss/v2/searches/b0b47d28-fce9-477b-ad761d1ca",
      "method": "GET"
    }
  },
  "searchId": "b0b47d28-fce9-477b-ad5b-951ea761d1ca",
  "save": false,
  "name": "MRN",
  "query": "clientReferenceInformation.code:TC50171_3987123 AND submitTimeUtc:[NOW/DAY TO NOW/DAY+1D
AY]",
  "count": 2,
  "totalCount": 2,
  "limit": 100,
  "offset": 0,
  "sort": "id:asc,submitTimeUtc:asc",
  "timezone": "America/Chicago",
  "submitTimeUtc": "2024-11-25T14:04:53Z",
  "_embedded": {
    "transactionSummaries": [
      {
        "id": "7325384341406368904951",
        "submitTimeUtc": "2024-11-25T12:40:34Z",
        "merchantId": "ahsamy",
        "applicationInformation": {
          "applications": [
            {
              "name": "ics_decision",
              "reasonCode": "100",
              "rCode": "1",
              "rFlag": "SOK",
              "reconciliationId": "76308783L301LIN2",
              "rMessage": "Service processed successfully",
              "returnCode": "1320000"
            }
          ]
        }
      }
    ]
  }
}
```

```

    },
    {
      "name": "ics_decision_early",
      "reasonCode": "100",
      "rCode": "1",
      "reconciliationId": "76308783L301LIN2",
      "returnCode": "9999999"
    },
    {
      "name": "ics_auth",
      "reasonCode": "100",
      "rCode": "1",
      "rFlag": "SOK",
      "reconciliationId": "76308783L301LIN2",
      "rMessage": "Request was processed successfully.",
      "returnCode": "1010000"
    }
  ],
  .
  .
  .
}
```

You will need to check the array "_embedded.transactionSummaries.Applications" and to check the status of "ics_auth"

Testing could be done using dev centre here [Search Transaction \(Dev Center\)](#)

Query filtration could be done using the following

- Query Parameter [Filtering by Query Parameter](#)
- paymentInformation.paymentType [Filtering Using paymentInformation.paymentType](#)
- submitTimeUtc [Filtering Using submitTimeUtc](#)

8.2. Retrieve transaction details

To retrieve transaction details using Cybersource RID, use the GET backend call
/tss/v2/transactions/{RID}

Example: GET /tss/v2/transactions/6709537307626466204951

Transaction details:

```

{
  "id": "6709537307626466204951",
  "rootId": "6709537307626466204951",
  "reconciliationId": "744806707BDH59TS",
  "submitTimeUTC": "2022-12-13T17:48:50Z",
  "merchantId": "25071980",
  "applicationInformation": {
    "reasonCode": 100,
    .
    .
    .
    "card": {
      "suffix": "1111",
      "expirationMonth": "11",
      "expirationYear": "2025",
      "type": "001"
    },
    "invoice": {},
    "accountFeatures": {},
    "fluidData": {}
  }
}
```

```
}  
},  
"tokenInformation": {},  
"_links": {  
  "self": {  
    "href": "https://apitest.cybersource.com/tss/v2/transactions/6709537307626466204951",  
    "method": "GET"  
  }  
}  
}
```

Could test retrieve transaction details using dev center here [Transaction Details | CyberSource Developer Center](#)

APPENDIX A: Useful Links

Glossary

[Cybersource glossary](#)

API Reference

[Cybersource REST API Reference](#)

REST API Dev Center:

[Cybersource REST API Reference](#)

Response Codes

[Transaction response codes | Cybersource Developer Center](#)

POSTMAN Collection

[Full list of POSTMAN samples](#)

Payer Authentication Test Cases:

[Payer Authentication Documentation](#)

GitHub Repositories

<https://github.com/CyberSource>

Reporting Fields

[Reporting Fields and Descriptions \(cybersource.com\)](#)

Reporting User guide

[Reporting user guide](#)

Apple Pay

<https://developer.cybersource.com/content/dam/docs/cybs/en-us/apple-pay/developer/ctv/rest/applepay.pdf>

Microform Guide:

<https://developer.cybersource.com/api/developer-guides/dita-flex/SAFlexibleToken/FlexMicroform.html>

Microform Full styling and events list in the guide:

<https://developer.cybersource.com/api/developer-guides/dita-flex/SAFlexibleToken/FlexMicroform/Styling.html>

Sample Code – Cybersource-flex-samples in .NET, JAVA, PHP and Node:

<https://github.com/CyberSource>

Webhook developer guide:

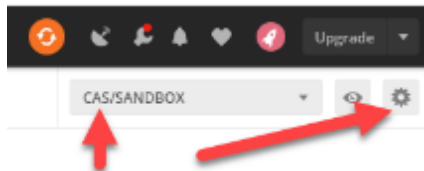
[Webhooks - Limited Availability](#)

APPENDIX B: Configure test MID in Postman

Step 1. Download the collection:

<https://documenter.getpostman.com/view/2960117/S17m1riA?version=latest#intro>

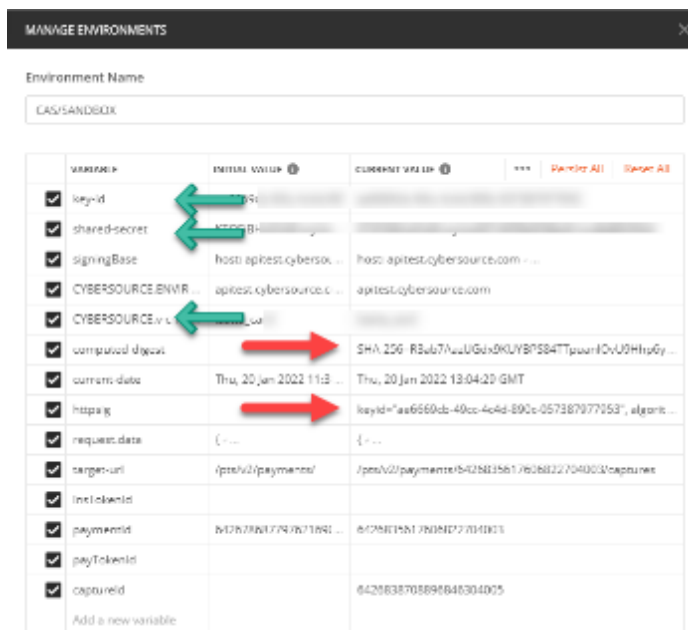
Step 2. Amend the Global Variables:



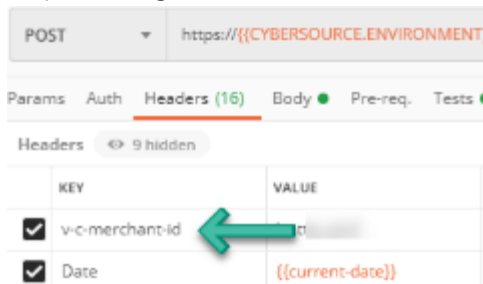
Step 3. Change the following variables:

Put your own key ID, Shared Secret and v-c-merchant ID.

Delete any prefilled values in computed-digest and httpsig so they are regenerated based on your details. Click UPDATE to save.



Step 4. Change the header value of each request to your own MID:



APPENDIX C: Data Types

Data Type	Permitted Characters and Formats
Alpha	Any letter from any language
AlphaNumeric	Alpha with any numeric character in any script
AlphaNumericPunctuation	Alphanumeric including ! " # \$ % & ' () * + , - . / : ; = ? @ ^ _ ~
Amount	0123456789 including a decimal point (.)
ASCIISAlphaNumericPunctuation	Any ASCII alphanumeric character including ! & ' () + , - . / : ; @
Date (a)	MM-YYYY
Date (b)	YYYYMMDD
Date (c)	yyyy-MM-dd HH:mm z yyyy-MM-dd hh:mm a z yyyy-MM-dd hh:mm z
Email	Valid standard email format. String (255) Customer's email address, including the full domain name. The field must be submitted in the form name@host.domain (for example, idoer@example.com). Email could contain letters, numbers, and special characters. Email couldn't contain spaces. For the Risk Update request, if the email address and the domain are sent in the request, the domain supersedes the email address. Email API validation regular expression is: <code>^[a-zA-Z0-9!#\$%&'*/+=?^_`{ }~ -]+[\.\a-zA-Z0-9!#\$%&'*/+=?^_`{ }~ -]*@[a-zA-Z0-9- _]+[a-zA-Z0-9- _]*\.[a-zA-Z]{2,15}\$</code>
Enumerated String	Comma-separated alphanumeric string
IP	Valid IP address
ISO 8601 Date	YYYY-MM-DDThh:mm:ssZ
Locale	[a-z] including a hyphen (-)
Numeric	0123456789
Phone	(), +, -, * # X 1234567890
URL	Valid URL (http or https)

APPENDIX D: MID Key Handling

We recommend handling Merchant ID Security Keys in the following manner:

- Limit read capabilities to the key files to only essential processes and personnel.
- Never leave key files in a location that can be scanned/served by a web application/server (i.e. leaving the keys exposed by Apache's HTTPD server).
- If keys must be transmitted electronically, utilize software like PGP encryption to encrypt the key contents during transmission.
- Keep automatic reminders scheduled to manage key expiration dates.
- Remember to generate separate keys for Production and Test; keys are non-transferable between the two environment types.

Notifications are sent 60 days before Security Keys expire, and regular notices are sent until the keys are updated. Please ensure that the Enterprise Business portal has up-to-date contact information for each Merchant ID account. It is recommended to use an alias/mailbox, rather than a specific user. Key expiration notifications will be sent to these contact email addresses.

APPENDIX E: Billing Default Data

Billing address data is mandatory for some Cybersource services and in case of absence of this data, you could use the below.

```
"billTo":  
{  
  "firstName": "John",  
  "lastName": "Doe",  
  "address1": "1 Market St",  
  "address2": "metro 3",  
  "locality": "san francisco",  
  "administrativeArea": "CA",  
  "postalCode": "94105",  
  "country": "US",  
  "email": "test@cybs.com",  
  "phoneNumber": "4158880000"  
}
```

APPENDIX F: Connect to Cybersource

Cybersource IPs Whitelisting:

In most cases, it's sufficient to add the **cybersource.com** domain to your Whitelist.

If, for some reason you need to add IP addresses, it's recommended that you add the full CyberSource IP range.

More details are available here [Knowledge Article | Cybersource IPs](#)

Please also check these articles

[Does CyberSource need to whitelist my IPs?](#)

[Where can I find the latest version of Cybersource server-level SSL certificates?](#)

[Where to find the recent CyberSource SSL certificates for ics2ws.ic3.com and ics2wstest.ic3.com?](#)